

前端面试题手册：Vue

基础知识

Vue的理解

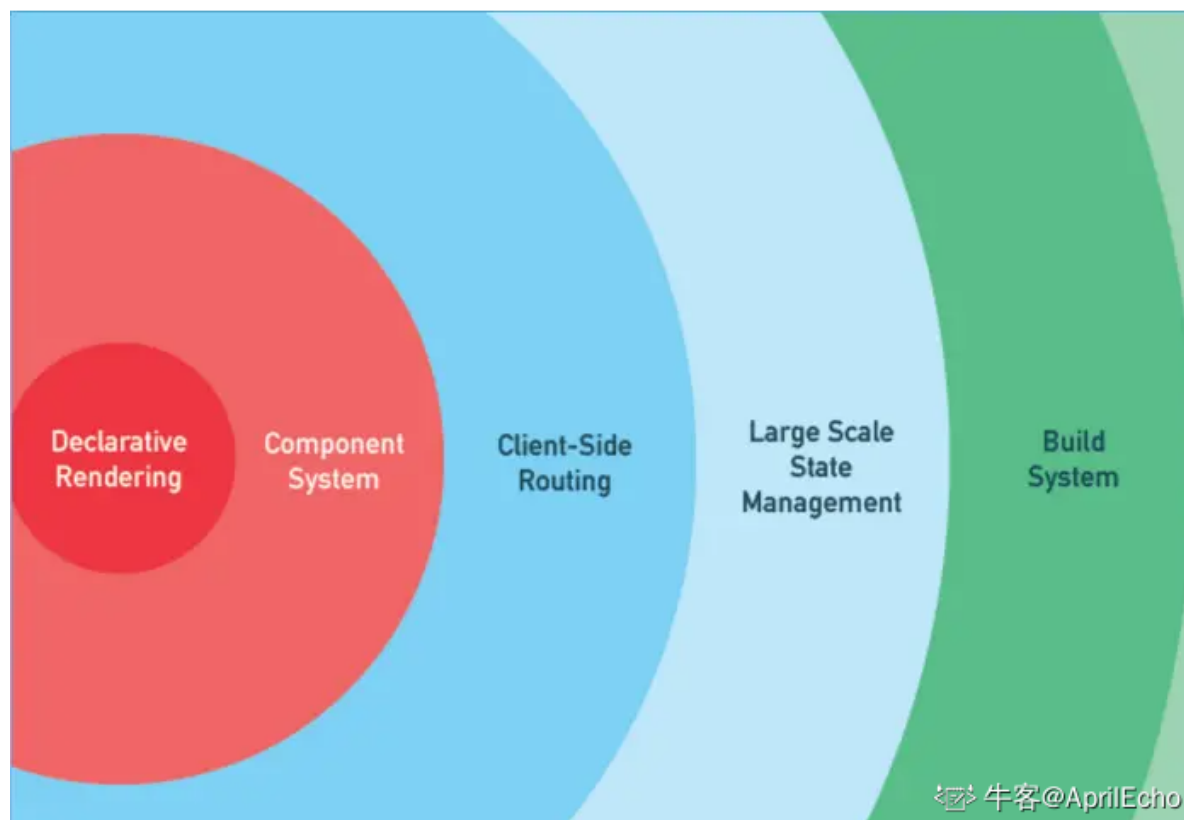
Vue是一个构建用户界面的渐进式框架，典型的 MVVM 框架。只关心图层；不关心具体是如何实现的。

Vue的优点主要有：

- 轻量级框架：只关视图层，是一个构建数据的视图集合，大小只有几十 kb ；
- 简单易学：国人开发，中文文档，不存在语言障碍，易于理解和学习；
- 双向数据绑定：保留了 angular 的特点，在数据操作方面更为简单；
- 组件化：保留了 react 的优点，实现了 html 的封装和重用，在构建单页面应用方面有着独特的优势；
- 视图，数据，结构分离：使数据的更改更为简单，不需要进行逻辑代码的修改，只需要操作数据就能完成相关操作；
- 虚拟DOM：dom 操作是非常耗费性能的，不再使用原生的 dom 操作节点，极大解放 dom 操作，但具体操作的还是 dom 不过是换了另一种方式；
- 运行速度更快：相比较于 react 而言，同样是操作虚拟 dom，就性能而言，vue 存在很大的优势。

为什么说Vue是一个渐进式框架

渐进式，通俗点讲就是，你想用啥你就用啥，咱也不强求你。你想用component就用，不用也行，你想用vuex就用，不用也可以。



组件的设计原则

(1)页面上每个独立的可视/可交互区域视为一个组件(比如页面的头部，尾部，可复用的区块)

(2)每个组件对应一个工程目录，组件所需要的各种资源在这个目录下就近维护(组件的就近维护思想体现了前端的工程化思想，为前端开发提供了很好的分治策略，在vue.js中，通过.vue文件将组件依赖的模板，js，样式写在一个文件中)

(每个开发者清楚开发维护的功能单元，它的代码必然存在在对应的组件目录中，在该目录下，可以找到功能单元所有的内部逻辑)

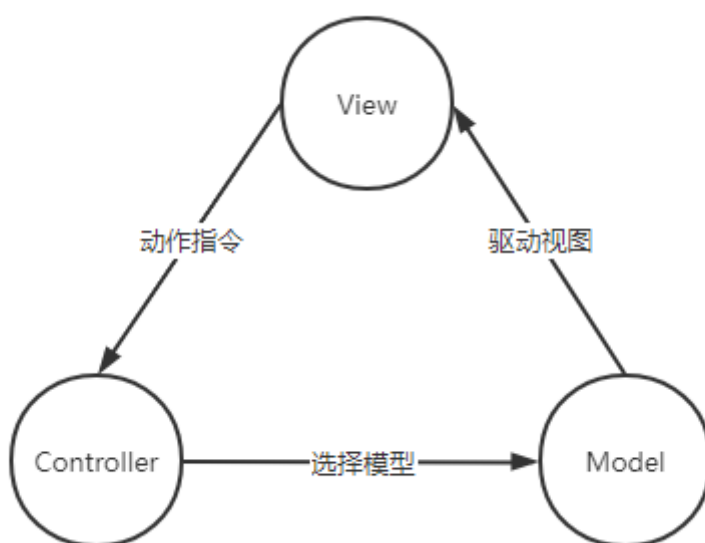
(3)页面不过是组件的容器，组件可以嵌套自由组合成完整的页面

vue.js 的两个核心是什么

数据驱动和组件化思想

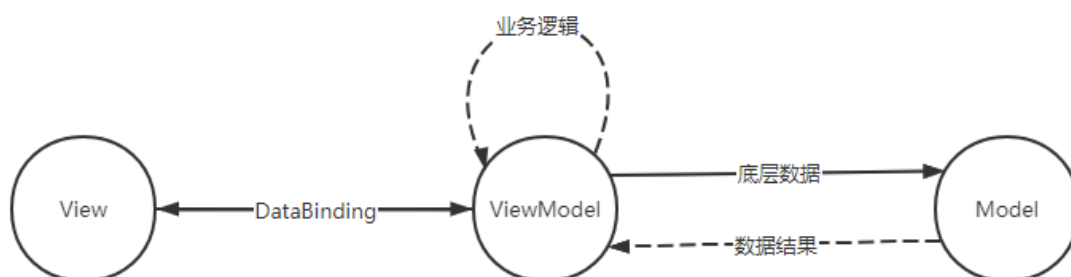
MVVM、MVC、MVP的区别

MVC



- MVC 通过分离 Model、View 和 Controller 的方式来组织代码结构。其中 View 负责页面的显示逻辑，Model 负责存储页面的业务数据，以及对相应数据的操作。并且 View 和 Model 应用了观察者模式，当 Model 层发生改变的时候它会通知有关 View 层更新页面。Controller 层是 View 层和 Model 层的纽带，它主要负责用户与应用的响应操作，当用户与页面产生交互的时候，Controller 中的事件触发器就开始工作了，通过调用 Model 层，来完成对 Model 的修改，然后 Model 层再去通知 View 层更新。

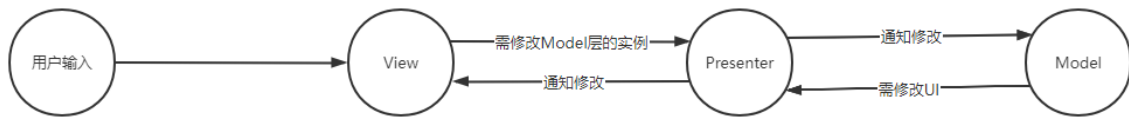
MVVM



MVVM 由 Model、View、ViewModel 三部分构成 Model 代表数据模型，也可以在 Model 中定义数据修改和业务逻辑；View 代表 UI 组件，它负责将数据模型转化成 UI 展现出来；ViewModel 是一个同步 View 和 Model 的对象；

- 对于 MVVM 来说，其实最重要的并不是通过双向绑定或者其他的方式将 View 与 ViewModel 绑定起来，而是通过 ViewModel 将视图中的状态和用户的行为分离出一个抽象，这才是 MVVM 的精髓。
- 在 MVVM 架构下，View 和 Model 之间并没有直接的联系，而是通过 ViewModel 进行交互，Model 和 ViewModel 之间的交互是双向的，因此 View 数据的变化会同步到 Model 中，而 Model 数据的变化也会立即反应到 View 上。
- ViewModel 通过双向数据绑定把 View 层和 Model 层连接了起来，而 View 和 Model 之间的同步工作完全是自动的，无需人为干涉，因此开发者只需关注业务逻辑，不需要手动操作 DOM，不需要关注数据状态的同步问题，复杂的数据状态维护完全由 MVVM 来统一管理。

MVP



- MVP 模式与 MVC 唯一不同的在于 Presenter 和 Controller。在 MVC 模式中使用观察者模式，来实现当 Model 层数据发生变化的时候，通知 View 层的更新。这样 View 层和 Model 层耦合在一起，当项目逻辑变得复杂的时候，可能会造成代码的混乱，并且可能会对代码的复用性造成一些问题。MVP 的模式通过使用 Presenter 来实现对 View 层和 Model 层的解耦。MVC 中的 Controller 只知道 Model 的接口，因此它没有办法控制 View 层的更新，MVP 模式中，View 层的接口暴露给了 Presenter 因此可以在 Presenter 中将 Model 的变化和 View 的变化绑定在一起，以此来实现 View 和 Model 的同步更新。这样就实现了对 View 和 Model 的解耦，Presenter 还包含了其他的响应逻辑。

MVVM的优缺点?

优点:

- 分离视图 (View) 和模型 (Model)，降低代码耦合，提高视图或者逻辑的重用性: 比如视图 (View) 可以独立于 Model 变化和修改，一个 ViewModel 可以绑定不同的 "View" 上，当 View 变化的时候 Model 不可以不变，当 Model 变化的时候 View 也可以不变。你可以把一些视图逻辑放在一个 ViewModel 里面，让很多 view 重用这段视图逻辑
- 提高可测试性: ViewModel 的存在可以帮助开发者更好地编写测试代码
- 自动更新 dom: 利用双向绑定, 数据更新后视图自动更新, 让开发者从繁琐的手动 dom 中解放

缺点:

- Bug 很难被调试: 因为使用双向绑定的模式，当你看到界面异常了，有可能是你 View 的代码有 Bug，也可能是 Model 的代码有问题。数据绑定使得一个位置的 Bug 被快速传递到别的位置，要定位原始出问题的地方就变得不那么容易了。另外，数据绑定的声明是指令式地写在 View 的模版当中的，这些内容是没办法去打断点 debug 的
- 一个大的模块中 model 也会很大，虽然使用方便了也很容易保证了数据的一致性，当长期持有，不释放内存就造成了花费更多的内存
- 对于大型的图形应用程序，视图状态较多，ViewModel 的构建和维护的成本都会比较高。

v-model 双向绑定的原理是什么

v-model 实际上是语法糖，下面就是语法糖的构造过程。

而 v-model 自定义指令下包裹的语法是 input 的 value 属性、input 事件，整个过程是：

```
<input v-model="inputV" />
// 等同于
<input :value="inputV" @input="inputV = $event.target.value"/>
```

- input属性绑定——inputV变量，也就是将值传给input；
- input事件，该事件在input的值inputV改变时触发，事件触发时给inputV重新赋值，所赋的值是 `$event.target.value`，也就是当前触发input事件对象的dom的value值，也就是该input的值。

这就完成了v-model的数据双向绑定。

我们会发现elementUI的所有自定义组件都适用v-model这一语法糖，除了input之外，select、textarea也用到这一语法糖。

- text和textarea元素：v-model使用value属性设置初始值并绑定变量，input事件更新值；
- checkbox和radio元素：v-model使用checked属性设置初始值并绑定变量，change事件更新值；
- select元素：v-model使用value属性设置初始值并绑定变量，change事件更新值；

比如checkbox：

```
// 看似执行了v-model一个指令
<input type="checkbox" v-model="checkedNames">
// 实际上
<input
  type="checkbox"
  :value="checkedNames"
  @change="checkedNames = $event.target.value"
/>
```

v-model 可以被用在自定义组件上吗？如果可以，如何使用

可以。v-model 实际上是一个语法糖，如：

```
<input v-model="searchText">
```

实际上相当于：

```
<input
  v-bind:value="searchText"
  v-on:input="searchText = $event.target.value"
>
```

用在自定义组件上也是同理：

```
<custom-input v-model="searchText">
```

相当于：

```
<custom-input
  v-bind:value="searchText"
  v-on:input="searchText = $event"
></custom-input>
```

显然，custom-input 与父组件的交互如下：

- 父组件将 searchText 变量传入custom-input 组件，使用的 prop 名为 value；
- custom-input 组件向父组件传出名为 input 的事件，父组件将接收到的值赋值给 searchText；

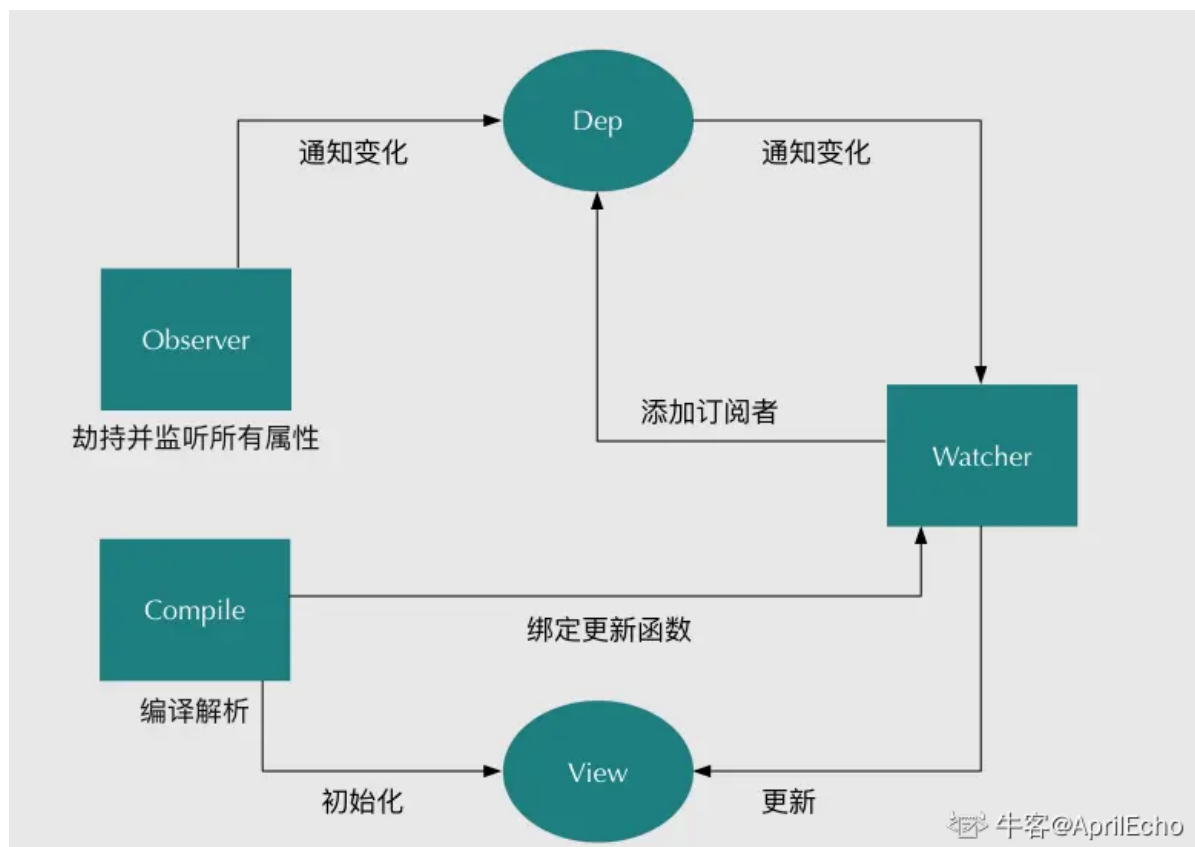
所以，custom-input 组件的实现应该类似于这样：

```
vue.component('custom-input', {
  props: ['value'],
  template: `
    <input
      v-bind:value="value"
      v-on:input="$emit('input', $event.target.value)"
    >
  `
})
```

讲一下Vue 2.0 响应式数据的原理

Vue.js 是采用**数据劫持**结合**发布者-订阅者模式**的方式，通过 Object.defineProperty()来劫持各个属性的 setter，getter，在数据变动时发布消息给订阅者，触发相应的监听回调。主要分为以下几个步骤：

- 需要 observe 的数据对象进行递归遍历，包括子属性对象的属性，都加上 setter 和 getter 这样的话，给这个对象的某个值赋值，就会触发 setter，那么就能监听到了数据变化；
- compile 解析模板指令，将模板中的变量替换成数据，然后初始化渲染页面视图，并将每个指令对应的节点绑定更新函数，添加监听数据的订阅者，一旦数据有变动，收到通知，更新视图；
- Watcher 订阅者是 Observer 和 Compile 之间通信的桥梁，主要做的事情是：①在自身实例化时往属性订阅器(dep)里面添加自己 ②自身必须有一个update()方法 ③待属性变动dep.notice()通知时，能调用自身的update()方法，并触发Compile中绑定的回调，则功成身退；
- MVVM 作为数据绑定的入口，整合 Observer、Compile 和 Watcher 三者，通过 Observer 来监听自己的 model 数据变化，通过 Compile 来解析编译模板指令，最终利用 Watcher 搭起 Observer 和 Compile 之间的通信桥梁，达到数据变化 -> 视图更新；视图交互变化(input) -> 数据 model 变更的双向绑定效果。



(其他) 使用 Object.defineProperty() 来进行数据劫持有什么缺点

- 在对一些属性进行操作时，使用这种方法无法拦截，比如通过下标方式修改数组数据或者给对象新增属性，这都不能触发组件的重新渲染，因为 `Object.defineProperty` 不能拦截到这些操作。更精确的来说，对于数组而言，大部分操作都是拦截不到的，只是 Vue 内部通过重写函数的方式解决了这个问题。
- 在 Vue3.0 中已经不使用这种方式了，而是通过使用 Proxy 对对象进行代理，从而实现数据劫持。使用 Proxy 的好处是它可以完美的监听到任何方式的数据改变，唯一的缺点是兼容性的问题，因为 Proxy 是 ES6 的语法。

Vue3.0 和 2.0 的响应式原理区别

Vue 在实例初始化时遍历 data 中的所有属性，并使用 `Object.defineProperty` 把这些属性全部转为 getter/setter。这样当追踪数据发生变化时，setter 会被自动调用。`Object.defineProperty` 是 ES5 中一个无法 shim 的特性，这也就是 Vue 不支持 IE8 以及更低版本浏览器的原因。

但是这样做有以下问题：

- 添加或删除对象的属性时，Vue 检测不到。因为添加或删除的对象没有在初始化进行响应式处理，只能通过 `$set` 来调用 `Object.defineProperty()` 处理。
- 无法监控到数组下标和长度的变化。

Vue3 使用 Proxy 来监控数据的变化。Proxy 是 ES6 中提供的功能，其作用为：用于定义基本操作的自定义行为（如属性查找，赋值，枚举，函数调用等）。相对于 `Object.defineProperty()`，其有以下特点：

- Proxy 直接代理整个对象而非对象属性，这样只需做一层代理就可以监听同级结构下的所有属性变化，包括新增属性和删除属性。
- Proxy 可以监听数组的变化。

Vue data 中某一个属性的值发生改变后，视图会立即同步执行重新渲染吗？

- 不会立即同步执行重新渲染。Vue 实现响应式并不是数据发生变化之后 DOM 立即变化，而是按一定的策略进行 DOM 的更新。Vue 在更新 DOM 时是异步执行的。只要侦听到数据变化，Vue 将开启一个队列，并缓冲在同一事件循环中发生的所有数据变更。
- 如果同一个 watcher 被多次触发，只会被推入到队列中一次。这种在缓冲时去除重复数据对于避免不必要的计算和 DOM 操作是非常重要的。然后，在下一个事件循环 tick 中，Vue 刷新队列并执行实际（已去重的）工作。

Vue 监控不了数组变化，有什么解决办法

当在项目中直接设置数组的某一项的值，或者直接设置对象的某个属性值，这个时候，你会发现页面并没有更新。这是因为 `Object.defineProperty()` 限制，监听不到变化。那么解决方式主要有几种方式：

- `this.$set(你要改变的数组/对象, 你要改变的位置/key, 你要改成什么value)`

```
this.$set(this.arr, 0, "OBKoro1"); // 改变数组
this.$set(this.obj, "c", "OBKoro1"); // 改变对象
```

- 调用以下几个数组的方法

```
splice()、push()、pop()、shift()、unshift()、sort()、reverse()
```

vue 源码里缓存了 array 的原型链，然后重写了这几个方法，触发这几个方法的时候会 observer 数据，意思是使用这些方法不用再进行额外的操作，视图自动进行更新。推荐使用 splice 方法***比较好自定义，因为 splice 可以在数组的任何位置进行删除/添加操作

`vm.$set` 的实现原理是：

- 如果目标是数组，直接使用数组的 `splice` 方法触发响应式；
- 如果目标是对象，会先判断属性是否存在、对象是否是响应式，最终如果要对属性进行响应式处理，则是通过调用 `defineReactive` 方法进行响应式处理（`defineReactive` 方法就是 Vue 在初始化对象时，给对象属性采用 `Object.defineProperty` 动态添加 getter 和 setter 的功能所调用的方法）

(其他) delete 和 Vue.delete 删除数组的区别

- `delete` 只是被删除的元素变成了 `empty/undefined` 其他的元素的键值还是不变。
- `Vue.delete` 直接删除了数组 改变了数组的键值。

Vue.set() 的原理

因为响应式数据 我们给对象和数组本身都增加了 `ob` 属性，代表的是 Observer 实例。当给对象新增不存在的属性 首先会把新的属性进行响应式跟踪 然后会触发对象 `ob` 的 `dep` 收集到的 watcher 去更新，当修改数组索引时我们调用数组本身的 `splice` 方法去更新数组。

相关代码如下：

```
export function set(target: Array | Object, key: any, val: any): any {
  // 如果是数组 调用我们重写的splice方法（这样可以更新视图）
  if (Array.isArray(target) && isValidArrayIndex(key)) {
    target.length = Math.max(target.length, key);
    target.splice(key, 1, val);
  }
```

```

    return val;
  }
  // 如果是对象本身的属性，则直接添加即可
  if (key in target && !(key in Object.prototype)) {
    target[key] = val;
    return val;
  }
  const ob = (target: any).__ob__;

  // 如果不是响应式的也不需要将其定义成响应式属性
  if (!ob) {
    target[key] = val;
    return val;
  }
  // 将属性定义成响应式的
  defineReactive(ob.value, key, val);
  // 通知视图更新
  ob.dep.notify();
  return val;
}

```

Vue 的单向数据流吗？

数据总是从父组件传到子组件，子组件没有权利修改父组件传过来的数据，只能请求父组件对原始数据进行修改。这样会防止从子组件意外改变父级组件的状态，从而导致你的应用的数据流向难以理解。

注意：在子组件直接用 v-model 绑定父组件传过来的 prop 这样是不规范的写法 开发环境会报警告

如果实在要改变父组件的 prop 值 可以再 data 里面定义一个变量 并用 prop 的值初始化它 之后用 \$emit 通知父组件去修改。

常见的事件修饰符及其作用

- `.stop`：等同于 JavaScript 中的 `event.stopPropagation()`，防止事件冒泡；
- `.prevent`：等同于 JavaScript 中的 `event.preventDefault()`，防止执行预设的行为（如果事件可取消，则取消该事件，而不停止事件的进一步传播）；
- `.capture`：与事件冒泡的方向相反，事件捕获由外到内；
- `.self`：只会触发自己范围内的事件，不包含子元素；
- `.once`：只会触发一次。

（其他）data 为什么是一个函数而不是对象

- JavaScript 中的对象是引用类型的数据，当多个实例引用同一个对象时，只要一个实例对这个对象进行操作，其他实例中的数据也会发生变化。
- 而在 Vue 中，更多的是想要复用组件，那就需要每个组件都有自己的数据，这样组件之间才不会相互干扰。
- 所以组件的数据不能写成对象的形式，而是要写成函数的形式。数据以函数返回值的形式定义，这样当每次复用组件的时候，就会返回一个新的 data，也就是说每个组件都有自己的私有数据空间，它们各自维护自己的数据，不会干扰其他组件的正常运行。

\$nextTick 原理及作用

Vue 的 nextTick 其本质是对 JavaScript 执行原理 EventLoop 的一种应用。

nextTick 的核心是利用了如 Promise、MutationObserver、setImmediate、setTimeout 的原生 JavaScript 方法来模拟对应的微/宏任务的实现，本质是为了利用 JavaScript 的这些异步回调任务队列来实现 Vue 框架中自己的异步回调队列。

nextTick 不仅是 Vue 内部的异步队列的调用方法，同时也允许开发者在实际项目中使用这个方法来满足实际应用中对 DOM 更新数据时机的后续逻辑处理

nextTick 是典型的将底层 JavaScript 执行原理应用到具体案例中的示例，引入异步更新队列机制的原因：

- 如果是同步更新，则多次对一个或多个属性赋值，会频繁触发 UI/DOM 的渲染，可以减少一些无用渲染
- 同时由于 VirtualDOM 的引入，每一次状态发生变化后，状态变化的信号会发送给组件，组件内部使用 VirtualDOM 进行计算得出需要更新的具体的 DOM 节点，然后对 DOM 进行更新操作，每次更新状态后的渲染过程需要更多的计算，而这种无用功也将浪费更多的性能，所以异步渲染变得更加至关重要

Vue 采用了数据驱动视图的思想，但是在一些情况下，仍然需要操作 DOM。有时候，可能遇到这样的情况，DOM1 的数据发生了变化，而 DOM2 需要从 DOM1 中获取数据，那这时就会发现 DOM2 的视图并没有更新，这时就需要用到了 nextTick 了。

由于 Vue 的 DOM 操作是异步的，所以，在上面的情况中，就要将 DOM2 获取数据的操作写在 `$nextTick` 中。

```
this.$nextTick(() => {    // 获取数据的操作...})
```

所以，在以下情况下，会用到 nextTick：

- 在数据变化后执行的某个操作，而这个操作需要使用随数据变化而变化的 DOM 结构的时候，这个操作就需要方法在 nextTick() 的回调函数中。
- 在 vue 生命周期中，如果在 created() 钩子进行 DOM 操作，也一定要放在 nextTick() 的回调函数中。

因为在 created() 钩子函数中，页面的 DOM 还未渲染，这时候也没办法操作 DOM，所以，此时如果想要操作 DOM，必须将操作的代码放在 nextTick() 的回调函数中。

Vue 是如何收集依赖的？

在初始化 Vue 的每个组件时，会对组件的 data 进行初始化，就会将由普通对象变成响应式对象，在这个过程中便会进行依赖收集的相关逻辑，如下所示：

```
function defineReactive (obj, key, val){
  const dep = new Dep();
  ...
  Object.defineProperty(obj, key, {
    ...
    get: function reactiveGetter () {
      if (Dep.target){
        dep.depend();
        ...
      }
      return val
    }
  })
  ...
}
```

以上只保留了关键代码，主要就是 `const dep = new Dep()` 实例化一个 Dep 的实例，然后在 get 函数中通过 `dep.depend()` 进行依赖收集。

- Dep

Dep是整个依赖收集的核心，其关键代码如下：

```
class Dep {
  static target;
  subs;

  constructor () {
    ...
    this.subs = [];
  }
  addSub (sub) {
    this.subs.push(sub)
  }
  removeSub (sub) {
    remove(this.sub, sub)
  }
  depend () {
    if(Dep.target){
      Dep.target.addDep(this)
    }
  }
  notify () {
    const subs = this.subs.slice();
    for(let i = 0; i < subs.length; i++){
      subs[i].update()
    }
  }
}
```

Dep 是一个 class，其中有一个关键的静态属性 static，它指向了一个全局唯一 Watcher，保证了同一时间全局只有一个 watcher 被计算，另一个属性 subs 则是一个 Watcher 的数组，所以 Dep 实际上就是对 Watcher 的管理，再看看 Watcher 的相关代码：

- Watcher

```
class Watcher {
  getter;
  ...
  constructor (vm, expression){
    ...
    this.getter = expression;
    this.get();
  }
  get () {
    pushTarget(this);
    value = this.getter.call(vm, vm)
    ...
    return value
  }
  addDep (dep){
    ...
    dep.addSub(this)
  }
}
```

```

    }
    ...
  }
  function pushTarget (_target) {
    Dep.target = _target
  }

```

Watcher 是一个 class，它定义了一些方法，其中和依赖收集相关的主要有 get、addDep 等。

- 过程

在实例化 Vue 时，依赖收集的相关过程如下：初始化状态 initState，这中间便会通过 defineReactive 将数据变成响应式对象，其中的 getter 部分便是用来依赖收集的。初始化最终会走 mount 过程，其中会实例化 Watcher，进入 Watcher 中，便会执行 this.get() 方法，

```

updateComponent = () => {
  vm._update(vm._render())
}
new watcher(vm, updateComponent)

```

get 方法中的 pushTarget 实际上就是把 Dep.target 赋值为当前的 watcher。

this.getter.call (vm, vm)，这里的 getter 会执行 vm.render() 方法，在这个过程中便会触发数据对象的 getter。那么每个对象值的 getter 都持有一个 dep，在触发 getter 的时候会调用 dep.depend() 方法，也就会执行 Dep.target.addDep(this)。刚才 Dep.target 已经被赋值为 watcher，于是便会执行 addDep 方法，然后走到 dep.addSub() 方法，便将当前的 watcher 订阅到这个数据持有的 dep 的 subs 中，这个目的是为后续数据变化时候能通知到哪些 subs 做准备。所以在 vm.render() 过程中，会触发所有数据的 getter，这样便已经完成了一个依赖收集的过程。

Vue 单页应用与多页应用的区别

概念：

- SPA 单页面应用 (SinglePage Web Application)，指只有一个主页面的应用，一开始只需要加载一次 js、css 等相关资源。所有内容都包含在主页面，对每一个功能模块组件化。单页应用跳转，就是切换相关组件，仅仅刷新局部资源。
- MPA 多页面应用 (MultiPage Application)，指有多个独立页面的应用，每个页面必须重复加载 js、css 等相关资源。多页应用跳转，需要整页资源刷新。

对比项/模式	SPA	MPA
结构	一个主页面+许多模块的组件	许多完整的页面
体验	页面切换快，体验佳；当初次加载文件过多时，需要做相关调优	页面切换慢，网速慢的时候，体验尤其不好
资源文件	组件公用的资源字需要加载一次	每个页面都要自己加载公用的资源
适用场景	对体验度和流畅度有较高要求的应用，不利于SEO（可借助SSR优化SEO）	适用于对SEO要求较高的应用
过渡动画	Vue提供了transition的封装组件，容易实现	很难实现
内容更新	相关组件的切换，即局部更新	整体HTML的切换，费钱（重复HTTP请求）
路由模式	可以使用hash，也可以使用history	普通链接跳转
数据传递	因为单页面，使用全局变量就好（Vuex）	cookie、localStorage等缓存方案，URL参数，调用接口保存等
相关成本	前期开发成本较高，后期维护较为容易	前期开发成本低，后期维护就比较麻烦，因为可能一个功能需要改很多地方

什么是 mixin ？

- mixin 使我们能够为 Vue 组件编写可插拔和可重用的功能。
- 如果希望在多个组件之间重用一组组件选项，例如生命周期 hook、方法等，则可以将其编写为 mixin，并在组件中简单的引用它。
- 然后将 mixin 的内容合并到组件中。如果你要在 mixin 中定义生命周期 hook，那么它在执行时将优化于组件自己的 hook。

mixin 和 mixins 区别

`mixin` 用于全局混入，会影响到每个组件实例，通常插件都是这样做初始化的。

```
vue.mixin({
  beforeCreate() { // ...逻辑 // 这种方式会影响到每个组件的
    beforeCreate 钩子函数  })
```

虽然文档不建议在应用中直接使用 `mixin`，但是如果不滥用的话也是很有帮助的，比如可以全局混入封装好的 `ajax` 或者一些工具函数等等。

`mixins` 应该是最常使用的扩展组件的方式了。如果多个组件中有相同的业务逻辑，就可以将这些逻辑剥离出来，通过 `mixins` 混入代码，比如上拉下拉加载数据这种逻辑等等。另外需要注意的是 `mixins` 混入的钩子函数会先于组件内的钩子函数执行，并且在遇到同名选项的时候也会有选择性的进行合并。

组件通信（重点）

组件通信的方式如下：

1、props / \$emit

父组件通过 props 向子组件传递数据，子组件通过 \$emit 和父组件通信

父组件向子组件传值

- props 只能是父组件向子组件进行传值，props 使得父子组件之间形成了一个单向下行绑定。子组件的数据会随着父组件不断更新。
- props 可以显示定义一个或一个以上的数据，对于接收的数据，可以是各种数据类型，同样也可以传递一个函数。
- props 属性名规则：若在 props 中使用驼峰形式，模板中需要使用短横线的形式

```
// 父组件
<template>
  <div id="father">
    <son :msg="msgData" :fn="myFunction"></son>
  </div>
</template>
```

```
//JavaScript代码
import son from "./son.vue";
export default {
  name: father,
  data() {
    msgData: "父组件数据";
  },
  methods: {
    myFunction() {
      console.log("vue");
    }
  },
  components: {
    son
  }
};
```

```
// 子组件
<template>
  <div id="son">
    <p>{{msg}}</p>
    <button @click="fn">按钮</button>
  </div>
</template>
```

```
//JavaScript代码
export default {
  name: "son",
  props: ["msg", "fn"]
};
```

子组件向父组件传值

- `$emit` 绑定一个自定义事件，当这个事件被执行的时候就会将参数传递给父组件，而父组件通过 `v-on` 监听并接收参数。

```
// 父组件
<template>
  <div class="section">
    <com-article :articles="articleList" @onEmitIndex="onEmitIndex"></com-article>
    <p>{{currentIndex}}</p>
  </div>
</template>

//JavaScript代码
import comArticle from './test/article.vue'
export default {
  name: 'comArticle',
  components: { comArticle },
  data() {
    return {
      currentIndex: -1,
      articleList: ['红楼梦', '西游记', '三国演义']
    }
  },
  methods: {
    onEmitIndex(idx) {
      this.currentIndex = idx
    }
  }
}

//子组件
<template>
  <div>
    <div v-for="(item, index) in articles" :key="index"
    @click="emitIndex(index)">{{item}}</div>
  </div>
</template>

//JavaScript代码
export default {
  props: ['articles'],
  methods: {
    emitIndex(index) {
      this.$emit('onEmitIndex', index) // 触发父组件的方法，并传递参数index
    }
  }
}
```

2、eventBus事件总线（`$emit` / `$on`）

`eventBus` 事件总线适用于父子组件、非父子组件等之间的通信，使用步骤如下：

- (1) 创建事件中心管理组件之间的通信

```
// event-bus.js

import Vue from 'vue'
export const EventBus = new Vue()
```

(2) 发送事件：假设有两个兄弟组件 firstCom 和 secondCom：

```
<template>
  <div>
    <first-com></first-com>
    <second-com></second-com>
  </div>
</template>

//JavaScript代码
import firstCom from './firstCom.vue'
import secondCom from './secondCom.vue'
export default {
  components: { firstCom, secondCom }
}
```

在 firstCom 组件中发送事件：

```
<template>
  <div>
    <button @click="add">加法</button>
  </div>
</template>

//JavaScript代码
import {EventBus} from './event-bus.js' // 引入事件中心

export default {
  data(){
    return{
      num:0
    }
  },
  methods:{
    add(){
      EventBus.$emit('addition', {
        num:this.num++
      })
    }
  }
}
```

(3) 接收事件：在 secondCom 组件中发送事件：

```
<template>
  <div>求和： {{count}}</div>
</template>
```

```
//JavaScript代码
import { EventBus } from './event-bus.js'
export default {
  data() {
    return {
      count: 0
    }
  },
  mounted() {
    EventBus.$on('addition', param => {
      this.count = this.count + param.num;
    })
  }
}
```

在上述代码中，这就相当于将 `num` 值存贮在了事件总线中，在其他组件中可以直接访问。事件总线就相当于一个桥梁，不用组件通过它来通信。

虽然看起来比较简单，但是这种方法也有不变之处，如果项目过大，使用这种方式进行通信，后期维护起来会很困难。

3、依赖注入 (provide / inject)

这种方式就是Vue中的**依赖注入**，该方法用于**父子组件之间的通信**。当然这里所说的父子不一定是真正的父子，也可以是祖孙组件，在层数很深的情况下，可以使用这种方法来进行传值。就不用一层一层的传递了。

`provide / inject` 是Vue提供的两个钩子，和 `data`、`methods` 是同级的。并且 `provide` 的书写形式和 `data` 一样。

- `provide` 钩子用来发送数据或方法
- `inject` 钩子用来接收数据或方法

在父组件中：

```
provide() {
  return {
    num: this.num
  };
}
```

在子组件中：

```
inject: ['num']
```

还可以这样写，这样写就可以访问父组件中的所有属性：

```

provide() {
  return {
    app: this
  };
}
data() {
  return {
    num: 1
  };
}

inject: ['app']
console.log(this.app.num)

```

注意： 依赖注入所提供的属性是**非响应式**的。

4、ref / \$refs

这种方式也是实现**父子组件**之间的通信。

ref： 这个属性用在子组件上，它的引用就指向了子组件的实例。可以通过实例来访问组件的数据和方法。

在子组件中：

```

export default {
  data () {
    return {
      name: 'JavaScript'
    }
  },
  methods: {
    sayHello () {
      console.log('hello')
    }
  }
}

```

在父组件中：

```

<template>
  <child ref="child"></component-a>
</template>

//JavaScript代码
import child from './child.vue'
export default {
  components: { child },
  mounted () {
    console.log(this.$refs.child.name); // JavaScript
    this.$refs.child.sayHello(); // hello
  }
}

```

5、parent / \$children

- 使用 `$parent` 可以让组件访问父组件的实例（访问的是上一级父组件的属性和方法）
- 使用 `$children` 可以让组件访问子组件的实例，但是，`$children` 并不能保证顺序，并且访问的数据也不是响应式的。

在子组件中：

```
<template>
  <div>
    <span>{{message}}</span>
    <p>获取父组件的值为:  {{parentVal}}</p>
  </div>
</template>

//JavaScript代码
export default {
  data() {
    return {
      message: 'vue'
    }
  },
  computed: {
    parentVal() {
      return this.$parent.msg;
    }
  }
}
```

在父组件中：

```
// 父组件中
<template>
  <div class="hello_world">
    <div>{{msg}}</div>
    <child></child>
    <button @click="change">点击改变子组件值</button>
  </div>
</template>

//JavaScript代码
import child from './child.vue'
export default {
  components: { child },
  data() {
    return {
      msg: 'welcome'
    }
  },
  methods: {
    change() {
      // 获取到子组件
      this.$children[0].message = 'JavaScript'
    }
  }
}
```

在上面的代码中，子组件获取到了父组件的 `parentVal` 值，父组件改变了子组件中 `message` 的值。需要注意：

- 通过 `$parent` 访问到的是上一级父组件的实例，可以使用 `$root` 来访问根组件的实例
- 在组件中使用 `$children` 拿到的是所有的子组件的实例，它是一个数组，并且是无序的
- 在根组件 `#app` 上拿 `$parent` 得到的是 `new vue()` 的实例，在这实例上再拿 `$parent` 得到的是 `undefined`，而在最底层的子组件拿 `$children` 是个空数组
- `$children` 的值是**数组**，而 `$parent` 是个**对象**

6. `attrs` / `$listeners`

考虑一种场景，如果A是B组件的父组件，B是C组件的父组件。如果想要组件A给组件C传递数据，这种隔代的数据，该使用哪种方式呢？

如果是用 `props/$emit` 来一级一级的传递，确实可以完成，但是比较复杂；如果使用事件总线，在多人开发或者项目较大的时候，维护起来很麻烦；如果使用Vuex，的确也可以，但是如果仅仅是传递数据，那可能就有点浪费了。

针对上述情况，Vue引入了 `$attrs` / `$listeners`，实现组件之间的跨代通信。

先来看一下 `inheritAttrs`，它的默认值`true`，继承所有的父组件属性除 `props` 之外的所有属性；

`inheritAttrs: false` 只继承`class`属性。

- `$attrs`：继承所有的父组件属性（除了`prop`传递的属性、`class` 和 `style`），一般用在子组件的子元素上
- `$listeners`：该属性是一个对象，里面包含了作用在这个组件上的所有监听器，可以配合 `v-on="$listeners"` 将所有的事件监听器指向这个组件的某个特定的子元素。（相当于子组件继承父组件的事件）

A组件（`APP.vue`）：

```
<template>
  <div id="app">
    //此处监听了两个事件，可以在B组件或者C组件中直接触发
    <child1 :p-child1="child1" :p-child2="child2" @test1="onTest1"
    @test2="onTest2"></child1>
  </div>
</template>

//JavaScript代码
import Child1 from './Child1.vue';
export default {
  components: { Child1 },
  methods: {
    onTest1() {
      console.log('test1 running');
    },
    onTest2() {
      console.log('test2 running');
    }
  }
};
```

B组件（`Child1.vue`）：

```
<template>
```

```

    <div class="child-1">
      <p>props: {{pChild1}}</p>
      <p>$attrs: {{$attrs}}</p>
      <child2 v-bind="$attrs" v-on="$listeners"></child2>
    </div>
  </template>

  //JavaScript代码
  import Child2 from './Child2.vue';
  export default {
    props: ['pChild1'],
    components: { Child2 },
    inheritAttrs: false,
    mounted() {
      this.$emit('test1'); // 触发APP.vue中的test1方法
    }
  };

```

C 组件 (Child2.vue):

```

<template>
  <div class="child-2">
    <p>props: {{pChild2}}</p>
    <p>$attrs: {{$attrs}}</p>
  </div>
</template>

//JavaScript代码
export default {
  props: ['pChild2'],
  inheritAttrs: false,
  mounted() {
    this.$emit('test2');// 触发APP.vue中的test2方法
  }
};

```

在上述代码中:

- C组件中能直接触发test的原因在于 B组件调用C组件时 使用 v-on 绑定了 \$listeners 属性
- 在B组件中通过v-bind 绑定 \$attrs 属性, C组件可以直接获取到A组件中传递下来的props (除了B组件中props声明的)

总结

(1) 父子组件间通信

- 子组件通过 props 属性来接受父组件的数据, 然后父组件在子组件上注册监听事件, 子组件通过 emit 触发事件来向父组件发送数据。
- 通过 ref 属性给子组件设置一个名字。父组件通过 \$refs 组件名来获得子组件, 子组件通过 \$parent 获得父组件, 这样也可以实现通信。
- 使用 provide/inject, 在父组件中通过 provide提供变量, 在子组件中通过 inject 来将变量注入到组件中。不论子组件有多深, 只要调用了 inject 那么就可以注入 provide中的数据。

(2) 兄弟组件间通信

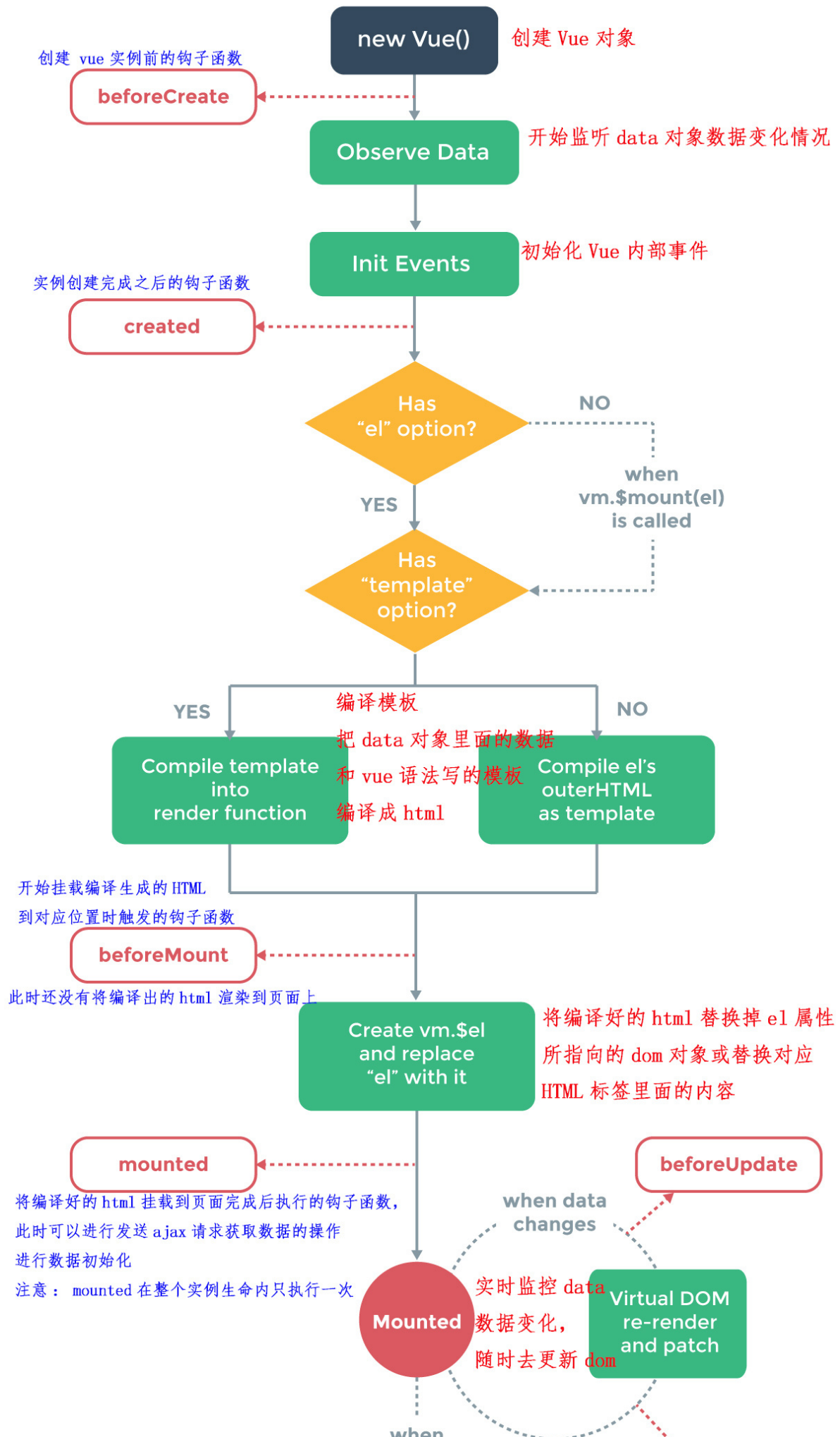
- 使用 eventBus 的方法，它的本质是通过创建一个空的 Vue 实例来作为消息传递的对象，通信的组件引入这个实例，通信的组件通过在这个实例上监听和触发事件，来实现消息的传递。
- 通过 `$parent/$refs` 来获取到兄弟组件，也可以进行通信。

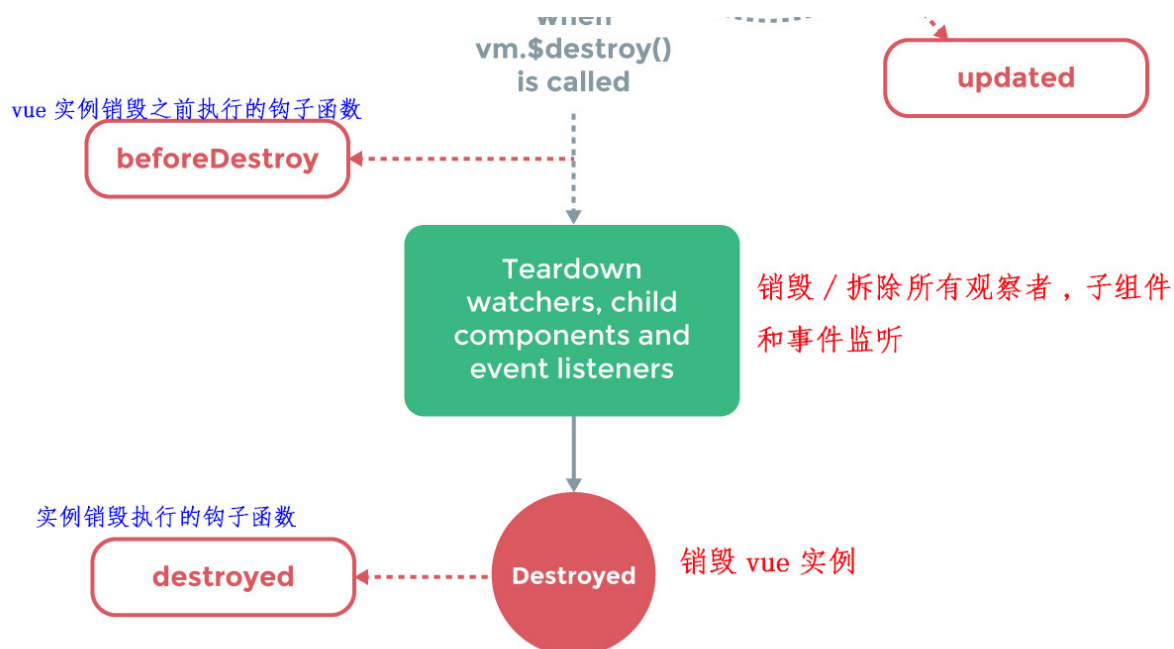
(3) 任意组件之间

- 使用 eventBus，其实就是创建一个事件中心，相当于中转站，可以用它来传递事件和接收事件。

如果业务逻辑复杂，很多组件之间需要同时处理一些公共的数据，这个时候采用上面这一些方法可能不利于项目的维护。这个时候可以使用 vuex，vuex 的思想就是将这一些公共的数据抽离出来，将它作为一个全局的变量来管理，然后其他组件就可以对这个公共数据进行读写操作，这样达到了解耦的目的。

Vue生命周期





Vue2 与Vue3的生命周期对比

Vue2	Vue3
beforeCreate(组件创建之前)	setup(组件创建之前)
created(组件创建完成)	setup(组件创建完成)
beforeMount(组件挂载之前)	onBeforeMount(组件挂载之前)
mounted(组件挂载完成)	onMounted(组件挂载完成)
beforeUpdate(数据更新，虚拟DOM打补丁之前)	onBeforeUpdate(数据更新，虚拟DOM打补丁之前)
updated(数据更新，虚拟DOM渲染完成)	onUpdated(数据更新，虚拟DOM渲染完成)
beforeDestroy(组件销毁之前)	onBeforeUnmount(组件销毁之前)
destroyed(组件销毁之后)	onUnmounted(组件销毁之后)

生命周期钩子函数

状态	说明
beforeCreate (创建前)	组件实例更被创建，组件属性计算之前，数据对象 data 都为 undefined，未初始化。
created (创建后)	组件实例创建完成，属性已经绑定，数据对象 data 已存在，但 dom 未生成，\$el 未存在
beforeMount (挂载前)	vue 实例的\$el 和 data 都已初始化，挂载之前为虚拟的 dom 节点，data.message 未替换
mounted (挂载后)	vue 实例挂载完成，data.message 成功渲染。
beforeUpdate (更新前)	当 data 变化时，会触发 beforeUpdate 方法
updated (更新后)	当 data 变化时，会触发 updated 方法
beforeDestroy (销毁前)	组件销毁之前调用
destroyed (销毁后)	组件销毁之后调用，对 data 的改变不会再触发周期函数，vue 实例已解除事件监听和 dom 绑定，但 dom 结构依然存在

Vue 子组件和父组件执行顺序

加载渲染过程：

1. 父组件 beforeCreate
2. 父组件 created
3. 父组件 beforeMount
4. 子组件 beforeCreate
5. 子组件 created
6. 子组件 beforeMount
7. 子组件 mounted
8. 父组件 mounted

更新过程：

1. 父组件 beforeUpdate
2. 子组件 beforeUpdate
3. 子组件 updated
4. 父组件 updated

销毁过程：

1. 父组件 beforeDestroy
2. 子组件 beforeDestroy
3. 子组件 destroyed
4. 父组件 destroyed

异步请求放在哪个生命周期中

我们可以在钩子函数 created、beforeMount、mounted 中进行调用，因为在这三个钩子函数中，data 已经创建，可以将服务端端返回的数据进行赋值。

推荐在 created 钩子函数中调用异步请求，因为在 created 钩子函数中调用异步请求有以下优点：

- 能更快获取到服务端数据，减少页面加载时间，用户体验更好；
- SSR不支持 beforeMount、mounted 钩子函数，放在 created 中有助于一致性。

keep-alive 中的生命周期哪些

keep-alive是 Vue 提供的一个内置组件，用来对组件进行缓存——在组件切换过程中将状态保留在内存中，防止重复渲染DOM。

如果为一个组件包裹了 keep-alive，那么它会多出两个生命周期：deactivated、activated。同时，beforeDestroy 和 destroyed 就不会再被触发了，因为组件不会被真正销毁。

当组件被换掉时，会被缓存到内存中、触发 deactivated 生命周期；当组件被切回来时，再去缓存里找这个组件、触发 activated钩子函数。

简单介绍keep-alive

- keep-alive 可以实现组件缓存，当组件切换时不会对当前组件进行卸载。
- 常用的两个属性 include/exclude，允许组件有条件的进行缓存。
- 两个生命周期 activated/deactivated，用来得知当前组件是否处于活跃状态。keep-alive 的中还运用了 LRU(Least Recently Used) 算法。

keep-alive 原理

- 在具体实现上，keep-alive 在内部维护了一个 key 数组和一个缓存对象

```
//keep-alive 内部声明周期函数
created () {
  this.cache = Object.create(null)
  this.keys = []
}
```

- key 数组记录目前缓存的组件 key 值，如果组件没有指定 key 值，会自动生成一个唯一的 key 值
- cache 对象会以 key 值为键，vnode 为值，用于缓存组件对应的虚拟 DOM
- 在 keep-alive 的渲染函数中，其基本逻辑是判断当前渲染的 vnode 是否有对应的缓存，如果有，会从缓存中读取到对应的组件实例，如果没有就会把它缓存。当缓存的数量超过 max 设置的数值时，keep-alive 会移除 key 数组中的第一个元素

拓展

- LRU缓存策略
 - 从内存中找出最久未使用的数据并置换新的数据。LRU (Least recently used) [算法](#)根据数据的历史访问记录来进行淘汰数据，其核心思想是 **"如果数据最近被访问过，那么将来被访问的几率也更高"**。
 - 利用map实现一个LRU[算法](#)

//力扣解题思路

1、创建map来保存数据

2、get: 访问某key，访问完要将其放在最后的。若key存在，先保存value值，删除key，再添加key，最后返回保存的value值。若key不存在，返回-1

3、put: 新加一个key，要将其放在最后的。所以，若key已经存在，先删除，再添加。如果容量超出范围了，将map中的头部删除。

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
  }
}
```

```

    this.map = new Map();
  }
  get(key) {
    if (this.map.has(key)) {
      // get表示访问该值
      // 所以在访问的同时，要将其调整位置，放置在最后
      const temp = this.map.get(key);
      // 先删除，再添加
      this.map.delete(key);
      this.map.set(key, temp);
      // 返回访问的值
      return temp;
    } else {
      // 不存在，返回-1
      return -1;
    }
  }
  put(key, value) {
    // 要将其放在最后，所以若存在key，先删除
    if (this.map.has(key)) this.map.delete(key);
    // 设置key、value
    this.map.set(key, value);
    if (this.map.size > this.capacity) {
      // 若超出范围，将map中头部的删除
      // map.keys()返回一个迭代器
      // 迭代器调用next()方法，返回包含迭代器返回的下一个值，在value中
      this.map.delete(this.map.keys().next().value);
    }
  }
}

```

(其他) created和mounted的区别

- created：在模板渲染成html前调用，即通常初始化某些属性值，然后再渲染成视图。
- mounted：在模板渲染成html后调用，通常是初始化页面完成后，再对html的dom节点进行一些需要的操作。

Vue路由

vue-router有多少种模式

- hash模式：兼容所有浏览器，包括不支持 HTML5 History Api 的浏览器，例如<http://www.baidu.com/#/index>，hash值为#/index，hash的改变会触发hashchange事件，我们可以通过监听hashchange事件来完成操作实现前端路由。

```

// 监听hash变化，点击浏览器的前进后退会触发
window.addEventListener('hashchange', function(event){
  let newURL = event.newURL; // hash 改变后的新 url
  let oldURL = event.oldURL; // hash 改变前的旧 url
}, false)

```

分析：当 URL 改变时，页面不会重新加载。hash (#) 是 URL 的锚点，代表的是网页中的一个位置，单单改变#后的部分，浏览器只会滚动到相应位置，不会重新加载网页，也就是说 #是用来指导浏览器动作的，对服务器端完全无用，HTTP请求中也不会不包括#；同时每一次改变#后的部分，都会在浏览器的访问历史中增加一个记录，使用“后退”按钮，就可以回到上一个位置；所以说

Hash模式通过锚点值的改变，根据不同的值，渲染指定DOM位置的不同数据

- history模式：能支持 HTML5 History Api 的浏览器，依赖HTML5 History API来实现前端路由。没有#的特殊符号，例如<http://www.baidu.com/index>，路由地址跟正常的url一样，不过第一次访问或者刷新页面都会向服务器请求，如果没有请求到对应的资源就会返回404，所以路由地址匹配不到任何静态资源，则应该返回同一个index.html 页面，需要在nginx中配置。

```
location / {
    try_files $uri $uri/ @router index index.html;
}
location @router {
    rewrite ^.*$ /index.html last;
}
```

分析：利用了 HTML5 History Interface 中新增的 pushState() 和 replaceState() 方法。pushState()方法可以改变URL地址且不会发送请求，replaceState()方法可以读取历史记录栈，还可以对浏览器记录进行修改。这两个方法应用于浏览器的历史记录栈，在当前已有的 back、forward、go 的基础之上，它们提供了对历史记录进行修改的功能。只是当它们执行修改时，虽然改变了当前的 URL，但浏览器不会立即向后端发送请求。

- abstract模式：支持所有 JavaScript 运行环境，如 Node.js 服务器端。如果发现没有浏览器的 API，路由会自动强制进入这个模式。

hash和history模式实现vue-router跳转api的区别

api	hash	history
push	window.location.assign	window.history.pushState
replace	window.location.replace	window.history.replaceState
go	window.history.go	window.history.go
back	window.history.go(-1)	window.history.go(-1)
forward	window.history.go(1)	window.history.go(1)

了解过动态路由吗

-	传参数	获取参数	url中形式	参数问题
this.\$route.query	this.\$router.push({path: 'index', query: {id: id}})	this.\$route.query.id	http://127.0.0.1:8080/#/index?id=1	刷新路由跳转页面参数不消失
this.\$route.params	this.\$router.push({name: 'index', params: {id: id}})	this.\$route.params.id	http://127.0.0.1:8080/#/index	刷新路由跳转页面参数消失

route和router的区别

1、router是VueRouter的一个对象，通过Vue.use(VueRouter)和Vue构造函数得到一个router的实例对象，这个对象中是一个全局的对象，他包含了所有的路由，包含了许多关键的对象和属性。

```

▼ VueRouter ⓘ
  ► afterHooks: []
  ► app: Vue {_uid: 2, _isVue: true, $options: {...}, _renderProxy: Proxy, _self: Vue, ...}
  ► apps: [Vue]
  ► beforeHooks: [f]
  ► fallback: false
  ► history: HashHistory {router: VueRouter, base: '', current: {...}, pending: {...}, ready: true, ...}
  ► matcher: {match: f, addRoutes: f}
  ► mode: "hash"
  ► options: {routes: Array(8), scrollBehavior: f}
  ► resolveHooks: []
  ► currentRoute: {...}
  ► [[Prototype]]: Object

```

2、route是一个跳转的路由对象，每一个路由都会有一个\$route对象，是一个局部的对象，可以获取对应的name, path, params, query等

```

▼ Object i
  fullPath: "/dashboard"
  hash: ""
  ▶ matched: (2) [{}], [{}]]
  ▶ meta: {__ob__: Observer}
  name: "Dashboard"
  ▶ params: {}
  path: "/dashboard"
  ▶ query: {}
  redirectedFrom: "/"
  ▶ [[Prototype]]: Object

```

从这两者不同的结构可以看出两者的区别，他们的一些属性是不同的

`$route.path` 字符串，等于当前路由对象的路径，会被解析为绝对路径，如/home/index

`$route.params` 对象，含路有动的动态片段和全匹配片段的键值对，不会拼接到路由的url后面

`$route.query` 对象，包含路由中查询参数的键值对。会拼接到路由url后面

`$route.router` 路由规则所属的路由器

`$route.matched` 数组，包含当前匹配的路径中所包含的所有片段所对象的配置参数对象

`$route.name` 当前路由的名字，如果没有使用具体路径，则名字为空

Vue-Router 的懒加载是如何实现的

- 这是一个普通的路由配置

```

import Vue from 'vue'
import Router from 'vue-router'
import HelloWorld from '@/components/HelloWorld'

Vue.use(Router)

export default new Router({
  routes: [
    {
      path: '/',
      name: 'HelloWorld',
      component: HelloWorld
    }
  ]
})

```

- vue异步组件实现懒加载

```

import Vue from 'vue'
import Router from 'vue-router'
/* 此处省去之前导入的HelloWorld模块 */
Vue.use(Router)

export default new Router({
  routes: [
    {
      path: '/',
      name: 'HelloWorld',
      //此处对原代码进行修改
      component: resolve=>(require(["@/components/HelloWorld"], resolve))
    }
  ]
})

```

```
]
})
```

- import方法

```
import Vue from 'vue'
import Router from 'vue-router'

Vue.use(Router)

export default new Router({
  routes: [
    {
      path: '/',
      name: 'HelloWorld',
      //此处对原代码进行修改
      component: ()=>import("@/components/HelloWorld")
    }
  ]
})
```

(拓展) 组件懒加载

- 这是一个普通的组件

```
<template>
  <div>
    <One></One>
  </div>
</template>

//JavaScript代码
import One from './one'
export default {
  components:{
    "One":One
  },
  data () {
    return {
      msg: 'This is a component'
    }
  }
}
```

- 异步方法

```
<template>
  <div>
    <One></One>
  </div>
</template>

//JavaScript代码
export default {
```

```

components:{
  //原代码修改
  "One": resolve=>(['./one'], resolve)
},
data () {
  return {
    msg: 'This is a component'
  }
}
}

```

- import方法

```

<template>
  <div>
    <One></One>
  </div>
</template>

//JavaScript代码
export default {
  components:{
    //原代码修改
    "One": ()=>import("./one");
  },
  data () {
    return {
      msg: 'This is a component'
    }
  }
}

```

路由导航守卫有哪些

- 全局守卫：beforeEach、beforeResolve、afterEach
- 路由独享守卫：beforeEnter
- 组件内的守卫：beforeRouteEnter、beforeRouteUpdate、beforeRouteLeave

路由守卫	名称	作用
全局守卫	beforeEach (to, from, next)	路由跳转前触发，常用于登录验证。
	beforeResolve (to, from, next)	在 beforeEach 和 组件内 beforeRouteEnter 之后，afterEach 之前调用。
	afterEach (to, from)	发生在 beforeEach 和 beforeResolve 之后，beforeRouteEnter 之前。路由在触发后执行。
路由独享守卫	beforeEnter	在 beforeEach 之后执行，和它功能一样，不怎么常用
组件内的守卫	beforeRouteEnter	路由进入之前调用。不能获取组件 this 实例，因为路由在进入组件之前，组件实例还没有被创建。
	beforeRouteUpdate	在当前路由改变时，并且该组件被复用时调用，可以通过 this 访问实例。当前路由 query 变更时，该守卫会被调用。
	beforeRouteLeave	导航离开该组件的对应路由时调用，可以访问组件实例 this。

导航守卫的三个参数

to：即将要进入的目标 路由对象。

from：当前导航正要离开的路由对象。

next：函数，必须调用，不然路由跳转不过去。

- `next()`：进入下一个路由。
- `next(false)`：中断当前的导航。
- `next('/')` 或 `next({ path: '/' })`：跳转到其他路由，当前导航被中断，进行新的一个导航。

触发钩子的完整顺序

- 路由导航、[keep-alive](#)、和组件生命周期钩子结合起来的，触发顺序，假设是从 a 组件离开，第一次进入 b 组件
 - 触发进入其他路由。
 - 调用要离开路由的组件守卫 `beforeRouteLeave`
 - 调用局前置守卫： `beforeEach`
 - 在重用的组件里调用 `beforeRouteUpdate`
 - 调用路由独享守卫 `beforeEnter`。
 - 解析异步路由组件。
 - 在将要进入的路由组件中调用 `beforeRouteEnter`
 - 调用全局解析守卫 `beforeResolve`
 - 导航被确认。
 - 调用全局后置钩子的 `afterEach` 钩子。
 - 触发DOM更新(`mounted`)。
 - 执行 `beforeRouteEnter` 守卫中传给 next 的回调函数

说说你对router-link的了解

`<router-link>` 是Vue-Router的内置组件，在具有路由功能的应用中作为声明式的导航使用。

`<router-link>` 有8个props，其作用是：

props	作用
to	必填，表示目标路由的链接。User
replace	默认值为false，若设置的话，当点击时，会调用 <code>router.replace()</code>
append	设置 append 属性后，则在当前 (相对) 路径前添加基路径。
tag	让渲染成tag设置的标签，如tag:'li',渲染结果为 foo
active-class	默认值为router-link-active,设置链接激活时使用的 CSS 类名。
exact-active-class	默认值为router-link-exact-active,设置链接被精确匹配的时候应该激活的 class。
exact	是否精确匹配，默认为false。
event	声明可以用来触发导航的事件。

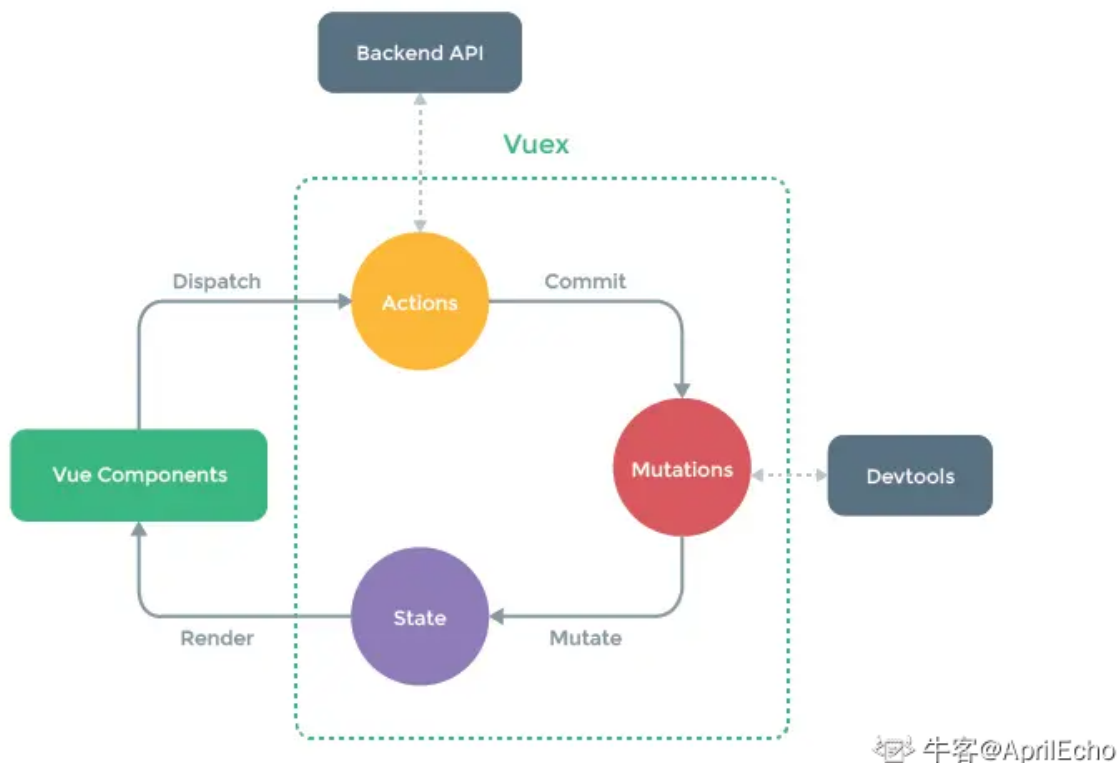
Vuex（重点）

vuex 的个人理解

vuex 是专门为 vue 提供的全局状态管理系统，用于多个组件中数据共享、数据缓存等。（无法持久化、内部核心原理是通过创建一个全局实例 new Vue）

主要包括以下几个模块：

- State：定义了应用状态的数据结构，可以在这里设置默认的初始状态。
- Getter：允许组件从 Store 中获取数据，mapGetters 辅助函数仅仅是将 store 中的 getter 映射到局部计算属性。
- Mutation：是唯一更改 store 中状态的方法，且必须是同步函数。
- Action：用于提交 mutation，而不是直接变更状态，可以包含任意异步操作。
- Module：允许将单一的 Store 拆分为多个 store 且同时保存在单一的状态树中。、



Vuex中action和mutation的区别

- action 提交的是 mutation，而不是直接变更状态。mutation 可以直接变更状态。
- action 可以包含任意异步操作。mutation 只能是同步操作。
- 提交方式不同，action 是用 `this.store.dispatch('ACTION_NAME',data)` 来提交。mutation 是用 `this.store.commit('SET_NUMBER',10)` 来提交。
- 接收参数不同，mutation 第一个参数是 state，而 action 第一个参数是 context

Redux 和 Vuex的区别

(1) Redux 和 Vuex区别

- Vuex改进了Redux中的Action和Reducer函数，以mutations变化函数取代Reducer，无需 switch，只需在对应的mutation函数里改变state值即可
- Vuex由于Vue自动重新渲染的特性，无需订阅重新渲染函数，只要生成新的State即可
- Vuex数据流的顺序是：View调用store.commit提交对应的请求到Store中对应的mutation函数->store改变（vue检测到数据变化自动渲染）

通俗点理解就是，vuex 弱化 dispatch，通过commit进行 store状态的一次变更;取消了action概念，不必传入特定的 action形式进行指定变更;弱化reducer，基于commit参数直接对数据进行转变，使得框架更加简易;

(2) 共同思想

- 单一的数据源
- 变化可以预测

本质上：redux与vuex都是对mwm思想的服务，将数据从视图中抽离的一种方案;形式上：vuex借鉴了redux，将store作为全局的数据中心，进行mode管理;

为什么要使用 Vuex 或者 Redux

由于传参的方法对于多层嵌套的组件将会非常繁琐，并且对于兄弟组件间的状态传递无能为力。我们经常会采用父子组件直接引用或者通过事件来变更和同步状态的多份拷贝。以上的这些模式非常脆弱，通常会导致代码无法维护。

所以需要把组件的共享状态抽取出来，以一个全局单例模式管理。在这种模式下，组件树构成了一个巨大的“视图”，不管在树的哪个位置，任何组件都能获取状态或者触发行为。

另外，通过定义和隔离状态管理中的各种概念并强制遵守一定的规则，代码将会变得更结构化且易维护。

为什么 Vuex 的 mutation 中不能做异步操作？

- Vuex中所有的状态更新的唯一途径都是mutation，异步操作通过 Action 来提交 mutation实现，这样可以方便地跟踪每一个状态的变化，从而能够实现一些工具帮助更好地了解我们的应用。
- 每个mutation执行完成后都会对应到一个新的状态变更，这样devtools就可以打个快照存下来，然后就可以实现 time-travel 了。如果mutation支持异步操作，就没有办法知道状态是何时更新的，无法很好的进行状态的追踪，给调试带来困难。

Vuex 页面刷新数据丢失怎么解决？

需要做 vuex 数据持久化 一般使用本地存储的方案来保存数据 可以自己设计存储方案 也可以使用第三方插件

推荐使用 vuex-persist 插件，它就是为 Vuex 持久化存储而生的一个插件。不需要你手动存取 storage，而是直接将状态保存至 cookie 或者 localStorage 中。（Redux同理）

Vuex 中 action 通常是异步的，那么如何知道 action 什么时候结束呢？

在 action 函数中返回 Promise，然后再提交时候用 then 处理。

在模块中，getter 和 mutation 和 action 中怎么访问全局的 state 和 getter？

- 在 getter 中可以通过第三个参数 rootState 访问到全局的 state,可以通过第四个参数 rootGetters 访问到全局的 getter。
- 在 mutation 中不可以访问全局的 state 和 getter，只能访问到局部的 state。
- 在 action 中第一个参数 context 中的 `context.rootState` 访问到全局的 state，`context.rootGetters` 访问到全局的 getter。

Vue指令

平时有用过哪些指令

- 主要有 `v-show`、`v-if`、`v-else-if`、`v-else`、`v-for`、`v-on`、`v-bind`、`v-model`、`v-once`、`v-slot`、`v-html`、`v-text`。

v-if、v-show、v-html 的原理

- v-if 会调用 `addIfCondition` 方法，生成 vnode 的时候会忽略对应节点，render 的时候就不会渲染；
- v-show 会生成 vnode，render 的时候也会渲染成真实节点，只是在 render 过程中会在节点的属性中修改 show 属性值，也就是常说的 display；

- v-html 会先移除节点下的所有节点，调用 html 方法，通过 addProp 添加 innerHTML 属性，归根结底还是设置 innerHTML 为 v-html 的值。

v-if 和 v-show 的区别

Vue 指令	v-if	v-show
共同点	动态显示DOM元素	动态设置DOM元素
手段	动态的向DOM树内添加或者删除DOM元素	设置DOM元素的display样式属性控制显示和隐藏
编译过程	有一个局部编译/卸载的过程，切换过程中合适地销毁和重建内部的事件监听和子组件	简单的基于CSS切换
编译条件	如果初始条件为假，则不进行操作；只有在条件第一次变为真时才开始局部编译	在任何条件下都会被编译，然后被缓存，而且DOM元素保留
性能消耗	v-if有更高的切换消耗	v-show有更高的初始渲染消耗
使用场景	v-if适合条件不太可能改变，也就是不需要频繁切换条件的场景	v-show适合频繁切换的场景

v-if和v-for中key的作用

vue 中 key 值的作用可以分为两种情况来考虑，

- 第一种情况是 v-if 中使用 key。由于 Vue 会尽可能高效地渲染元素，通常会复用已有元素而不是从头开始渲染。因此当使用 v-if 来实现元素切换的时候，如果切换前后含有相同类型的元素，那么这个元素就会被复用。如果是相同的 input 元素，那么切换前后用户的输入不会被清除掉，这样是不符合需求的。因此可以通过使用 key 来唯一的标识一个元素，这个情况下，使用 key 的元素不会被复用。这个时候 key 的作用是用来标识一个独立的元素。
- 第二种情况是 v-for 中使用 key。用 v-for 更新已渲染过的元素列表时，它默认使用“就地复用”的策略。如果数据项的顺序发生了改变，Vue 不会移动 DOM 元素来匹配数据项的顺序，而是简单复用此处的每个元素。因此通过为每个列表项提供一个 key 值，来以便 Vue 跟踪元素的身份，从而高效的实现复用。这个时候 key 的作用是为了高效的更新渲染虚拟 DOM。

题外话：当然，在开发过程并非一定要使用key，如果只是为了简单展示数据，其实也可以index来标识，视情况而定就好啦。

v-for 与 v-if 的优先级

- 1、v-for优先于v-if被解析；
- 2、如果同时出现，每次渲染都会先执行循环再判断条件，无论如何循环都不可避免，浪费了性能；
- 3、要避免出现这种情况，则在外层嵌套template，在这一层进行v-if判断，然后在内部进行v-for循环；
- 4、如果条件出现在循环内部，可通过计算属性提前过滤掉那些不需要显示的项。

为什么不建议用index作为key?

使用index 作为 key和没写基本上没区别，因为不管数组的顺序怎么颠倒，index 都是 0, 1, 2...这样排列，导致 Vue 会复用错误的旧子节点，做很多额外的工作。

为什么避免 v-if 和 v-for 同时使用

vue2.x 中v-for优先级高于v-if，vue3.x 相反。所以2.x 版本中在一个元素上同时使用 v-if 和 v-for 时，v-for 会优先作用，造成性能浪费；3.x 版本中 v-if 总是优先于 v-for 生效，导致v-if访问不了v-for中的变量。

解析：

一般我们在两种常见的情况下会倾向于这样做：

- 为了过滤一个列表中的项目 (比如 v-for="user in users" v-if="user.isActive")。在这种情形下，请将 users 替换为一个计算属性 (比如 activeUsers)，让其返回过滤后的列表。
- 为了避免渲染本应该被隐藏的列表 (比如 v-for="user in users" v-if="shouldShowUsers")。这种情形下，请将 v-if 移动至容器元素上 (比如 ul、ol)。

当 Vue 处理指令时，v-for 比 v-if 具有更高的优先级，所以这个模板：

```
<ul>
  <li
    v-for="user in users"
    v-if="user.isActive"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>
```

将会经过如下运算：

```
this.users.map(function (user) {
  if (user.isActive) {
    return user.name
  }
})
```

因此哪怕我们只渲染出一小部分用户的元素，也得在每次重渲染的时候遍历整个列表，不论活跃用户是否发生了变化。

通过将其更换为在如下的一个计算属性上遍历：

```
computed: {
  activeUsers: function () {
```

```

        return this.users.filter(function (user) {
            return user.isActive
        })
    }
}
<ul>
  <li
    v-for="user in activeUsers"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>

```

我们将会获得如下好处：

- 过滤后的列表只会在 users 数组发生相关变化时才被重新运算，过滤更高效。
- 使用 v-for="user in activeUsers" 之后，我们在渲染的时候只遍历活跃用户，渲染更高效。
- 解耦渲染层的逻辑，可维护性 (对逻辑的更改和扩展) 更强。

为了获得同样的好处，我们也可以把：

```

<ul>
  <li
    v-for="user in users"
    v-if="shouldShowUsers"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>

```

更新为：

```

<ul v-if="shouldShowUsers">
  <li
    v-for="user in users"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>

```

通过将 v-if 移动到容器元素，我们不会再对列表中的每个用户检查 shouldShowUsers。取而代之的是，我们只检查它一次，且不会在 shouldShowUsers 为否的时候运算 v-for。

反例：

```

<ul>
  <li
    v-for="user in users"
    v-if="user.isActive"
    :key="user.id"
  >
    {{ user.name }}
  </li>

```

```
</ul>

<ul>
  <li
    v-for="user in users"
    v-if="shouldShowUsers"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>
```

正确例子

```
<ul>
  <li
    v-for="user in activeUsers"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>

<ul v-if="shouldShowUsers">
  <li
    v-for="user in users"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>
```

v-on可以绑定多个方法吗

<p v-on="{click:one,mousemove:two}">v-on绑定多个方法</p>这里绑定一个点击事件和鼠标移动事件

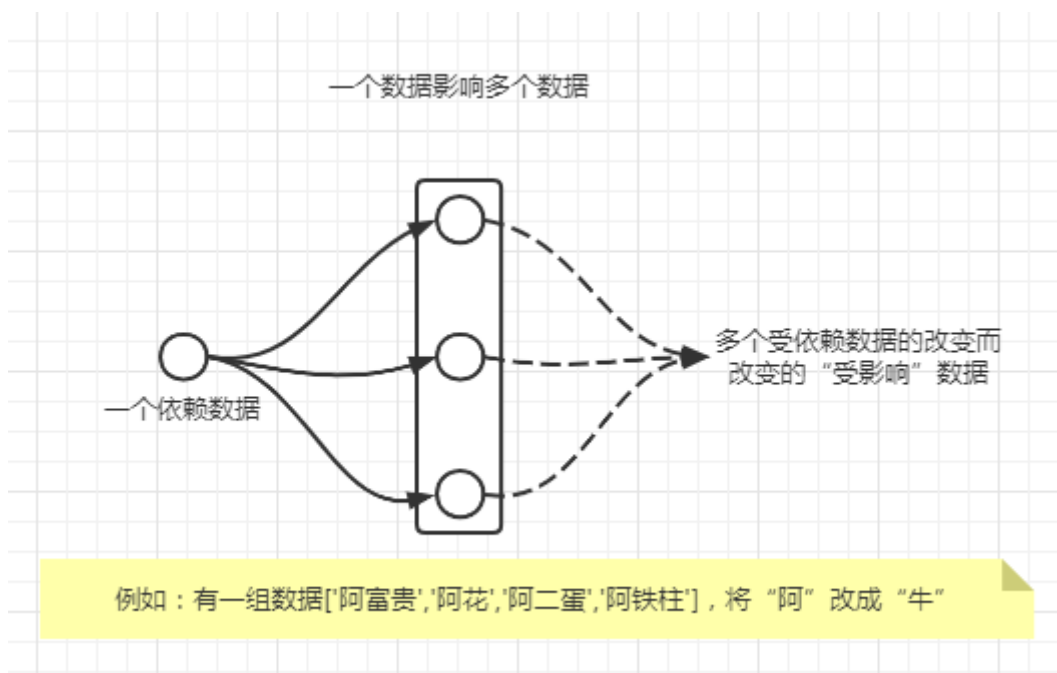
Vue属性

methods watch和compute的区别

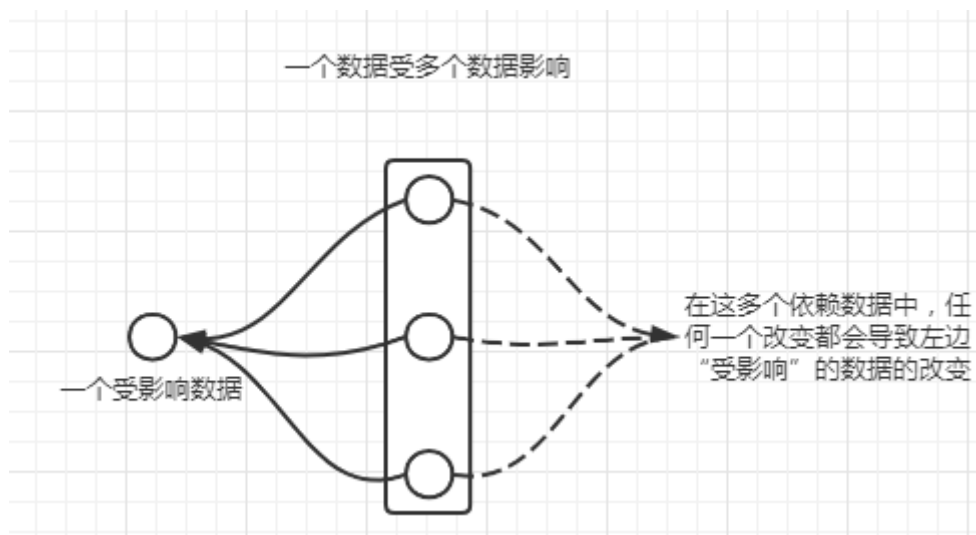
-	computed	watch	methods
作用机制	自动调用，完成我们希望完成的作用	自动调用，完成我们希望完成的作用	主动调用
性质	计算属性，事实上和data对象里的数据属性是相同的	类似于监听机制跟事件机制	定义的是函数，使用时跟函数调用一样
缓存	支持缓存，只有依赖的数据发生了变化，才会重新计算	不支持缓存，数据变化时，它就会触发相应的操作	无
是否支持异步	不支持异步，当 Computed 中有异步操作时，无法监听数据的变化	支持异步监听	支持异步处理
场景	一个数据受多个数据影响	一个数据影响多个数据	提供可调用的函数

- computed和methods关于缓存的一个小细节
 - 在computed中定义一个计算属性，并且返回new Date()，可以发现多次返回的时间是相同的。这是因为new Date()不是依赖型数据（不是放在data对象下的实例数据），所以computed只提供了缓存的值，而没有重新计算。这也是整理这方面的知识过程发现的一个小细节。
- watch和computed处理场景的对比

watch



computed



虚拟DOM（重点/加分项）

讲一下Virtual DOM

由于在浏览器中操作 DOM 是很昂贵的。频繁操作 DOM，会产生一定性能问题。这就是虚拟 Dom 的产生原因。Vue2 的 Virtual DOM 借鉴了开源库 snabbdom 的实现。Virtual DOM 本质就是用一个原生的 JS 对象去描述一个 DOM 节点，是对真实 DOM 的一层抽象。

优点：

- 保证性能下限：框架的虚拟 DOM 需要适配任何上层 API 可能产生的操作，他的一些 DOM 操作的实现必须是普适的，所以它的性能并不是最优的；但是比起粗暴的 DOM 操作性能要好很多，因此框架的虚拟 DOM 至少可以保证在你不需要手动优化的情况下，依然可以提供还不错的性能，既保证性能的下限。
- 无需手动操作 DOM：我们不需手动去操作 DOM，只需要写好 View-Model 的代码逻辑，框架会根据虚拟 DOM 和数据双向绑定，帮我们以可预期的方式更新视图，极大提高我们的开发效率。
- 跨平台：虚拟 DOM 本质上是 JavaScript 对象，而 DOM 与平台强相关，相比之下虚拟 DOM 可以进行更方便地跨平台操作，例如服务器端渲染、weex 开发等等。

缺点：

- 无法进行极致优化：虽然虚拟 DOM + 合理的优化，足以应对大部分应用的性能需要，但在一些性能要求极高的应用中虚拟 DOM 无法进行针对性的极致优化。
- 首次渲染大量 DOM 时，由于多了一层 DOM 计算，会比 innerHTML 插入慢。

了解diff算法吗

- 首先，对比节点本身，判断是否为同一节点，如果不为相同节点，则删除该节点重新创建节点进行替换
- 如果为相同节点，进行patchVnode，判断如何对该节点的子节点进行处理，先判断一方有子节点一方没有子节点的情况(如果新的children没有子节点，将旧的子节点移除)
- 比较如果都有子节点，则进行updateChildren，判断如何对这些新老节点的子节点进行操作（diff 核心）。
- 匹配时，找到相同的子节点，递归比较子节点

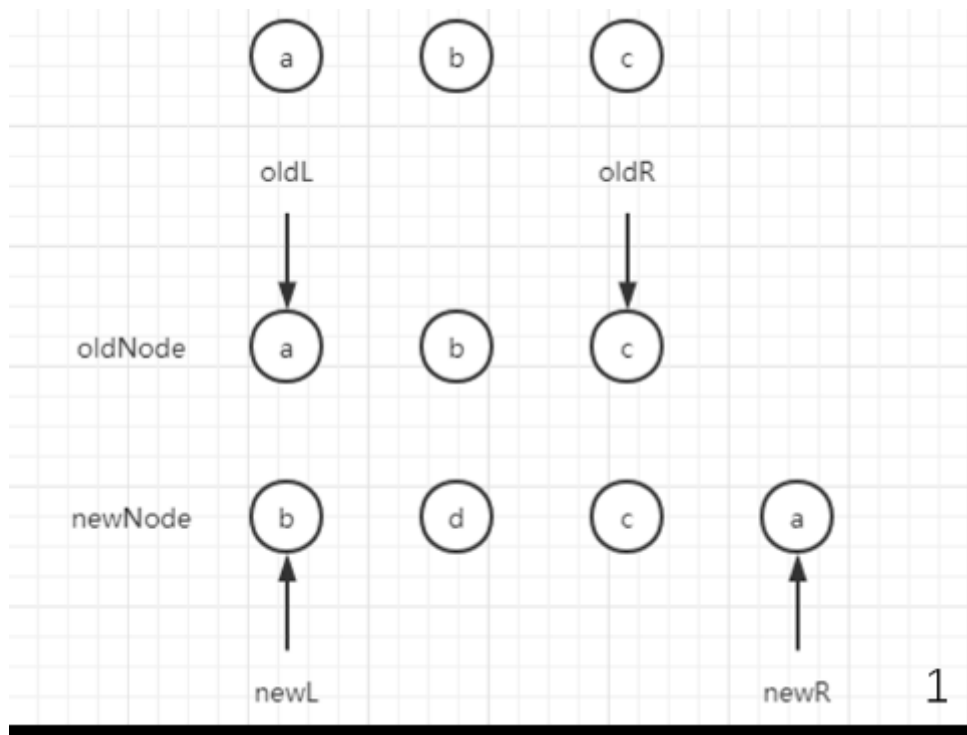
拓展

patchVnode函数做了哪些操作

- 找到对应的 真实DOM，称为 e1
- 判断 newVnode 和 oldVnode 是否指向同一个对象，如果是，那么直接 return

- 如果他们都有文本节点并且不相等，那么将 e1 的文本节点设置为 newVnode 的文本节点。
- 如果 oldVnode 有子节点而 newVnode 没有，则删除 e1 的子节点
- 如果 oldVnode 没有子节点而 newVnode 有，则将 newVnode 的子节点真实化之后添加到 e1
- 如果两者都有子节点，则执行 updateChildren 函数比较子节点，这一步很重要

updateChildren方法



另外，如果一开始oldL在newNode的指针找不到时，新列表的第一个节点b去旧列表进行遍历比较，这里会有两种情况，找到相同节点 和 没找到相同节点

找到的情况，在旧节点中找到相同节点b，将节点b移动到首位，然后重新开始进行双端的步骤对比。如果在旧节点找不到，则在头部直接添加新节点，并将newL指针指向下一位，再进行对比。

拓展

(其他) Vue 3.0

Vue3.0有什么更新

(1) 监测机制的改变

- 3.0 将带来基于代理 Proxy 的 observer 实现，提供全语言覆盖的反应性跟踪。
- 消除了 Vue 2 当中基于 Object.defineProperty 的实现所存在的很多限制：

(2) 只能监测属性，不能监测对象

- 检测属性的添加和删除；
- 检测数组索引和长度的变更；
- 支持 Map、Set、WeakMap 和 WeakSet。

(3) 模板

- 作用域插槽，2.x 的机制导致作用域插槽变了，父组件会重新渲染，而 3.0 把作用域插槽改成了函数的方式，这样只会影响子组件的重新渲染，提升了渲染的性能。

- 同时，对于 render 函数的方面，vue3.0 也会进行一系列更改来方便习惯直接使用 api 来生成 vdom。

(4) 对象式的组件声明方式

- vue2.x 中的组件是通过声明的方式传入一系列 option，和 TypeScript 的结合需要通过一些装饰器的方式来做，虽然能实现功能，但是比较麻烦。
- 3.0 修改了组件的声明方式，改成了类式的写法，这样使得和 TypeScript 的结合变得很容易

(5) 其它方面的更改

- 支持自定义渲染器，从而使得 weex 可以通过自定义渲染器的方式来扩展，而不是直接 fork 源码来改的方式。
- 支持 Fragment（多个根节点）和 Portal（在 dom 其他部分渲染组建内容）组件，针对一些特殊的场景做了处理。
- 基于 tree shaking 优化，提供了更多的内置功能。

Vue3.0 为什么要用 proxy?

在 Vue2 中，Object.defineProperty 会改变原始数据，而 Proxy 是创建对象的虚拟表示，并提供 set、get 和 deleteProperty 等处理器，这些处理器可在访问或修改原始对象上的属性时进行拦截，有以下特点：

- 不需用使用 `vue.$set` 或 `vue.$delete` 触发响应式。
- 全方位的数组变化检测，消除了Vue2 无效的边界情况。
- 支持 Map, Set, WeakMap 和 WeakSet。

Proxy 实现的响应式原理与 Vue2的实现原理相同，实现方式大同小异：

- get 收集依赖
- Set、delete 等触发依赖
- 对于集合类型，就是对集合对象的方法做一层包装：原方法执行后执行依赖相关的收集或触发逻辑。

了解过Vue插槽吗，有几种？

slot 又名插槽，是 Vue 的内容分发机制，组件内部的模板引擎使用 slot 元素作为承载分发内容的出口。插槽 slot 是子组件的一个模板标签元素，而这一个标签元素是否显示，以及怎么显示是由父组件决定的。slot 又分三类，默认插槽，具名插槽和作用域插槽。

- 默认插槽：又名匿名插槽，当 slot 没有指定 name 属性值的时候一个默认显示插槽，一个组件内只有有一个匿名插槽，作为找不到匹配的内容片段时的备用插槽。
- 具名插槽：带有具体名字的插槽，也就是带有 name 属性的 slot，一个组件可以出现多个具名插槽。
- 作用域插槽：默认插槽、具名插槽的一个变体，可以是匿名插槽，也可以是具名插槽，该插槽的不同点是在子组件渲染作用域插槽时，可以将子组件内部的数据传递给父组件，让父组件根据子组件的传递过来的数据决定如何渲染该插槽。

实现原理：当子组件 vm 实例化时，获取到父组件传入的 slot 标签的内容，存放在 `vm.$slots` 中，默认插槽为 `vm.$slots.default`，具名插槽为 `vm.$slots.xxx`，xxx 为插槽名，当组件执行渲染函数时候，遇到 slot 标签，使用 `$slots` 中的内容进行替换，此时可以为插槽传递数据，若存在数据，则可称该插槽为作用域插槽。

(其他) 对 React 和 Vue 的理解，它们的异同

相似之处：

- 都将注意力集中保持在核心库，而将其他功能如路由和全局状态管理交给相关的库；
- 都有自己的构建工具，能让你得到一个根据最佳实践设置的项目模板；
- 都使用了Virtual DOM（虚拟DOM）提高重绘性能；
- 都有props的概念，允许组件间的数据传递；
- 都鼓励组件化应用，将应用分拆成一个个功能明确的模块，提高复用性。

不同之处：

1) 数据流

Vue默认支持数据双向绑定，而React一直提倡单向数据流

2) 虚拟DOM

Vue2.x开始引入"Virtual DOM"，消除了和React在这方面的差异，但是在具体的细节还是有各自的特点。

- Vue宣称可以更快地计算出Virtual DOM的差异，这是由于它在渲染过程中，会跟踪每一个组件的依赖关系，不需要重新渲染整个组件树。
- 对于React而言，每当应用的状态被改变时，全部子组件都会重新渲染。当然，这可以通过PureComponent/shouldComponentUpdate这个生命周期方法来进行控制，但Vue将此视为默认的优化。

3) 组件化

React与Vue最大的不同是模板的编写。

- Vue鼓励写近似常规HTML的模板。写起来很接近标准HTML元素，只是多了一些属性。
- React推荐你所有的模板通用JavaScript的语法扩展——JSX书写。

具体来讲：React中render函数是支持闭包特性的，所以import的组件在render中可以直接调用。但是在Vue中，由于模板中使用的数据都必须挂在this上进行一次中转，所以import一个组件完了之后，还需要在components中再声明下。

4) 监听数据变化的实现原理不同

- Vue通过getter/setter以及一些函数的劫持，能精确知道数据变化，不需要特别的优化就能达到很好的性能
- React默认是通过比较引用的方式进行的，如果不优化（PureComponent/shouldComponentUpdate）可能导致大量不必要的vDOM的重新渲染。这是因为Vue使用的是可变数据，而React更强调数据的不可变。

5) 高阶组件

react可以通过高阶组件（HOC）来扩展，而Vue需要通过mixins来扩展。

高阶组件就是高阶函数，而React的组件本身就是纯粹的函数，所以高阶函数对React来说易如反掌。相反Vue.js使用HTML模板创建视图组件，这时模板无法有效的编译，因此Vue不能采用HOC来实现。

6) 构建工具

两者都有自己的构建工具：

- React ==> Create React APP
- Vue ==> vue-cli

7) 跨平台

- React ==> React Native
- Vue ==> Weex

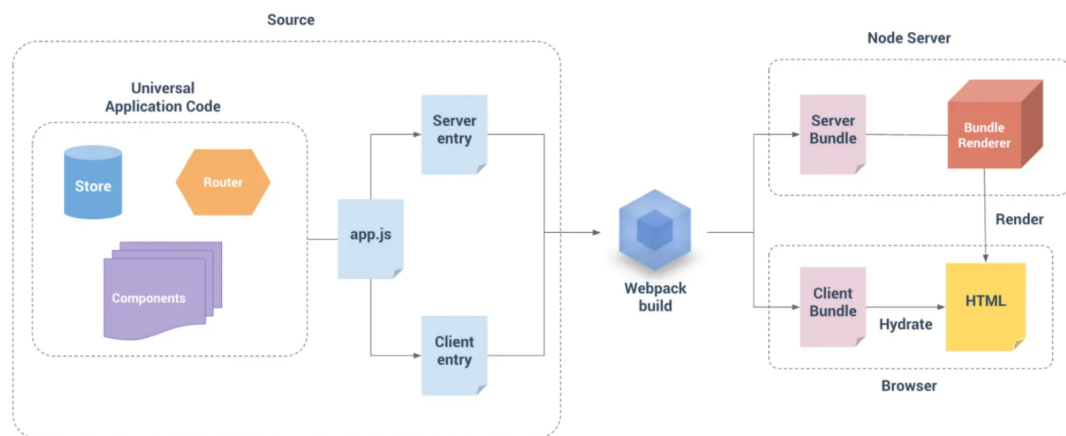
有做过哪些性能优化？

- 对象层级不要过深，否则性能就会差。
- 不需要响应式的数据不要放在 data 中（可以使用 Object.freeze() 冻结数据）
- v-if 和 v-show 区分使用场景
- computed 和 watch 区分场景使用
- 大数据列表和表格性能优化——虚拟列表 / 虚拟表格
- 防止内部泄露，组件销毁后把全局变量和时间销毁
- 服务端渲染
- 图片懒加载
- 路由懒加载
- 适当采用 [keep-alive](#) 缓存组件
- 开启 gzip 压缩
- 防抖、节流的运用

服务端渲染了解过吗？

- 服务端渲染（SSR），也就是将 vue 在客户端把标签渲染成 HTML 的工作放在服务端完成，然后再把 html 直接返回给客户端。
- SSR 有着更好的 SEO、并且首屏加载速度更快等优点。不过它也有一些缺点，比如我们的开发条件会受到限制，服务器端渲染只支持 beforeCreate 和 created 两个钩子，当我们需要一些外部扩展库时需要特殊处理，服务端渲染应用程序也需要处于 Node.js 的运行环境。还有就是服务器会有更大的负载需求。

大致流程就是将 Source（源码）通过 webpack 打包出两个 bundle，其中 Server Bundle 是给服务端用的，服务端通过渲染器 bundleRenderer 将 bundle 生成 html 给浏览器用；另一个 Client Bundle 是给浏览器用的，别忘了服务端只是生成前期首屏页面所需的 html，后期的交互和数据处理还是需要能支持浏览器脚本的 Client Bundle 来完成。



@掘金技术社区

一个小栗子

```
// 第 1 步: 创建一个 vue 实例
const Vue = require('vue')
const app = new Vue({
  template: `<div>Hello world</div>`
})
// 第 2 步: 创建一个 renderer
const renderer = require('vue-server-renderer').createRenderer()
// 第 3 步: 将 vue 实例渲染为 HTML
renderer.renderToString(app, (err, html) => {
  if (err) throw err
  console.log(html)
  // => <div data-server-rendered="true">Hello world</div>
})
```

上面例子利用 `vue-server-renderer` npm 包将一个vue示例最后渲染出了一段 html。将这段html发送给客户端就轻松的实现了服务器渲染了。

```
const server = require('express')()
server.get('*', (req, res) => {
  // ... 生成 html
  res.end(html)
})
server.listen(8080)
```

服务端渲染和客户端渲染的区别

-	客户端渲染	服务端渲染
html的生成原理	由js生成html	由后台语言通过一些模板引擎生成
优点	前端做视图和交互	响应快，用户体验好
	后端提供接口，数据	搜索引擎友好，有seo优化
	前后端分离	nodejs层服务器渲染，前端性能优化更顺手
	前端做路由	可操作空间更大
	服务器计算压力变轻	
缺点	用户等待时间变长，尤其是请求数多且有一定先后顺序的时候	增加服务器计算压力；如果不是增加node中间层，前后端分工不明，不能很好的并行并发
耗时比较	数据请求：客户端在不同网络环境进行数据请求，外网http请求开销大，导致时间差	数据请求：服务端在内网请求，数据响应速度快
	步骤：客户端需要等待js代码下载，加载完成在请求数据，渲染	步骤：服务端是先请求数据再渲染可视化部分，即服务端不需要等待js代码下载，并会返回一个已经有内容的页面
	渲染内容：客户端渲染，是经历一个从无到有完整的渲染步骤	渲染内容：服务端先渲染可视化部分，客户端再做二次渲染
适合场景	单页面应用，如Vue	用户体验比较高的比如首屏加载，重复较多的公共页面可以使用服务器渲染，减少ajax请求，提高用户体验

讲讲图片懒加载

- 图片懒加载大概思路，渲染时设置一个节点的自定义属性，比如说 `data-src`，然后值为图片 url 地址，图片的 `src` 属性指向懒加载的封面，监听 `scroll` 事件，通过 `getClientBoundingRect` API 获得图片相对视口的位置，当图片距离视口底部一定时，替换 url 地址。达成目标；
- 当浏览器支持 `Intersection Observer` API 时，可以使用该构造函数创建一个观察者，观察所有待懒加载的图片资源；
- 现在浏览器原生支持图片和 `iframe` 懒加载，使用 `loading="lazy"`，不过不太可控，而且浏览器兼容性并不好。

```
<img v-lazy="/static/img/01.png"/>
```

gzip 压缩了解多少

gzip压缩是一种http请求优化方式，通过减少文件体积来提高加载速度。html、js、css文件甚至json数据都可以用它压缩，可以减小60%以上的体积。前端配置gzip压缩，并且服务端使用nginx开启gzip，用来减小网络传输的流量大小。

牛富贵：命令行执行：`npm i compression-webpack-plugin -D`

牛富贵：在webpack的dev开发配置文件中加入如下代码：

```
const CompressionWebpackPlugin = require('compression-webpack-plugin')
plugins: [
  new CompressionWebpackPlugin()
]
```

启用gzip压缩打包之后，会自动生成gz包。目前大部分主流浏览器客户端都是支持gzip的，不支持gzip格式文件的会默认访问源文件的，故不要配置清除源文件。配置好之后，打开浏览器访问线上，F12查看控制台，如果该文件资源的响应头里显示有Content-Encoding: gzip，表示浏览器支持并且启用了Gzip压缩的资源。

实现一个axios拦截器

- 新建 request.js 文件,导入 axios

新建 request.js 文件,导入 axios

- 创建一个 axios 的实例

```
const request = axios.create({
  baseURL: xxx,
  // baseURL: '项目基地址'
  timeout: 5000 // 设置 5 秒延时关闭请求
})
```

- 设置请求拦截器

```
// request.interceptors.request.use() // 请求拦截器
request.interceptors.request.use(config => {

  config.headers.Authorization = `Bearer ${token}` // 设置请求头携带 token
  return config
}, error => {
  console.log(error) // 发生错误打印
  return error
})
```

- 设置响应拦截器

```
// request.interceptors.response.use() // 响应拦截器
request.interceptors.response.use(config => {
  return config // 成功直接返回
}, error => {
  if (error.response.status === 401) { //如果发生错误,查看错误码是多少 401 为权限不够,token 过期
    alert('token 请求超时!请重新登录!')
    // 进行操作,如删除 vuex 中过期用户数据等一系列操作
    router.push('/login') // 强行返回到登录页
  }
  return error
})
```

- 导出 axios 实例

```
export default request
```

如何中断axios请求

官方提供了两种方法

- 使用 `CancelToken.source` 工厂方法创建 cancel token

```
const CancelToken = axios.CancelToken;
const source = CancelToken.source();

axios.get('/user/12345', {
  cancelToken: source.token
}).catch(function(thrown) {
  if (axios.isCancel(thrown)) {
    console.log('Request canceled', thrown.message);
  } else {
    // 处理错误
  }
});

axios.post('/user/12345', {
  name: 'new name'
}, {
  cancelToken: source.token
})

// 取消请求（message 参数是可选的）
source.cancel('operation canceled by the user.');
```

- 通过传递一个 executor 函数到 `CancelToken` 的构造函数来创建 cancel token

```
const CancelToken = axios.CancelToken;
let cancel;

axios.get('/user/12345', {
  cancelToken: new CancelToken(function executor(c) {
    // executor 函数接收一个 cancel 函数作为参数
    cancel = c;
  })
});

// cancel the request
cancel();
```

Vue渲染大量数据时应该怎么优化

对于长列表，通常分两种情况来优化。

- 一是静态列表：如果这个列表仅仅用于数据的展示，不会有任何数据变化，那么就不需要作响应式处理。但由于 vue 中 data 是响应式的，所以我们可以利用 `Object.freeze` 将其冻结起来。

```
export default {
  data: () => {
    return {
```

```

    users: [
      /* a long static list */
    ]
  };
},
async create() {
  const users = await axios.get("/users");
  //数据冻结
  this.users = Object.freeze(users);
}
};

```

`Object.freeze()` 方法可以冻结一个对象，冻结指的是不能向这个对象添加新的属性，不能修改其已有属性的值，不能删除已有属性，以及不能修改该对象已有属性的可枚举性、可配置性、可写性。该方法返回被冻结的对象。

```

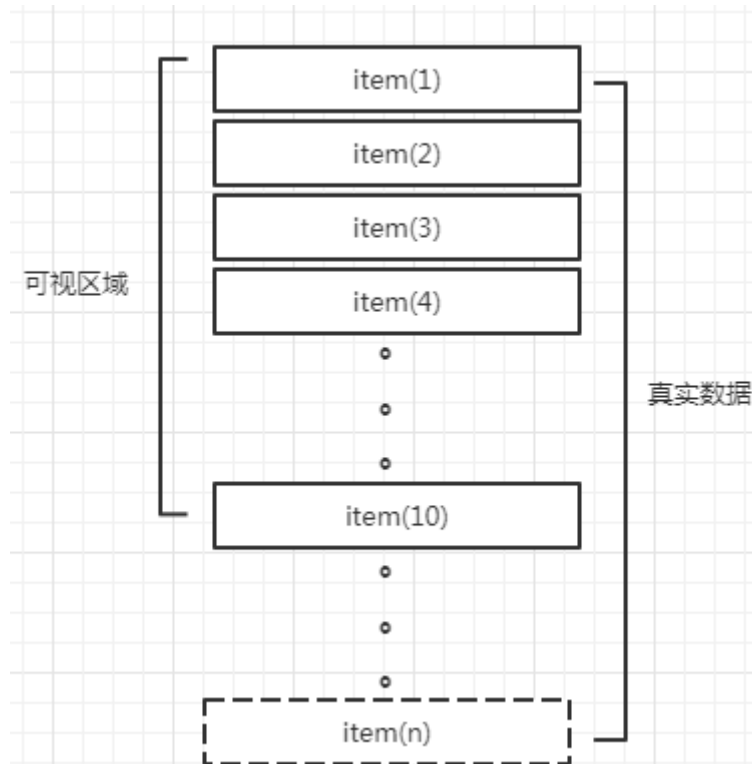
//举个栗子
let arr = [0];
Object.freeze(a); // 数组中数据不能被修改了。

arr[0] = 1; // fails silently
arr.push(2); // fails silently

```

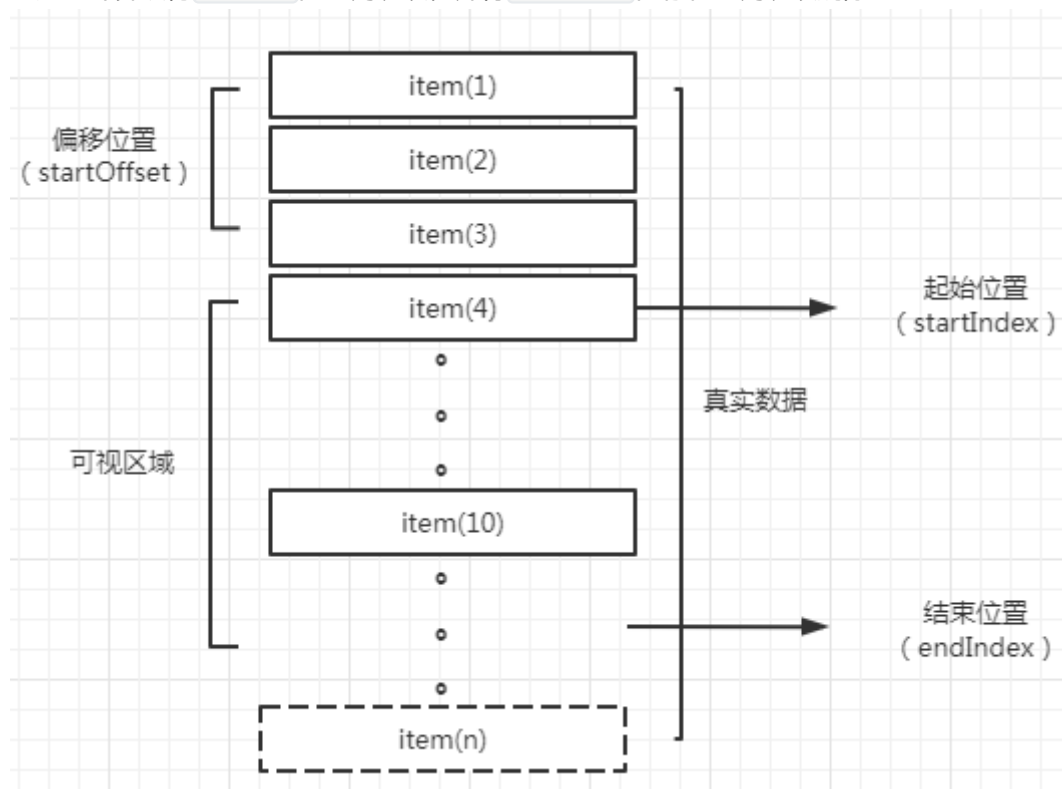
- 二是虚拟滚动：对于大数据的长列表，如果一次性全部渲染，显然是非常消耗性能的，所以可以采用虚拟滚动技术，只渲染被展示出来的部分

1、假设有 1 万条记录需要同时渲染，我们屏幕的可见区域的高度为 500px，而列表项的高度为 50px，则此时我们在屏幕中最多只能看到 10 个列表项，那么在首次渲染的时候，我们只需加载 10 条即可。



- 2、当滚动发生时，我们可以通过计算当前滚动值得知此时在屏幕可见区域应该显示的列表项。
- 3、假设滚动发生，滚动条距顶部的位置为 150px，则我们可得知在可见区域内的列表项为第 4 项至第 13 项。
- 4、实现

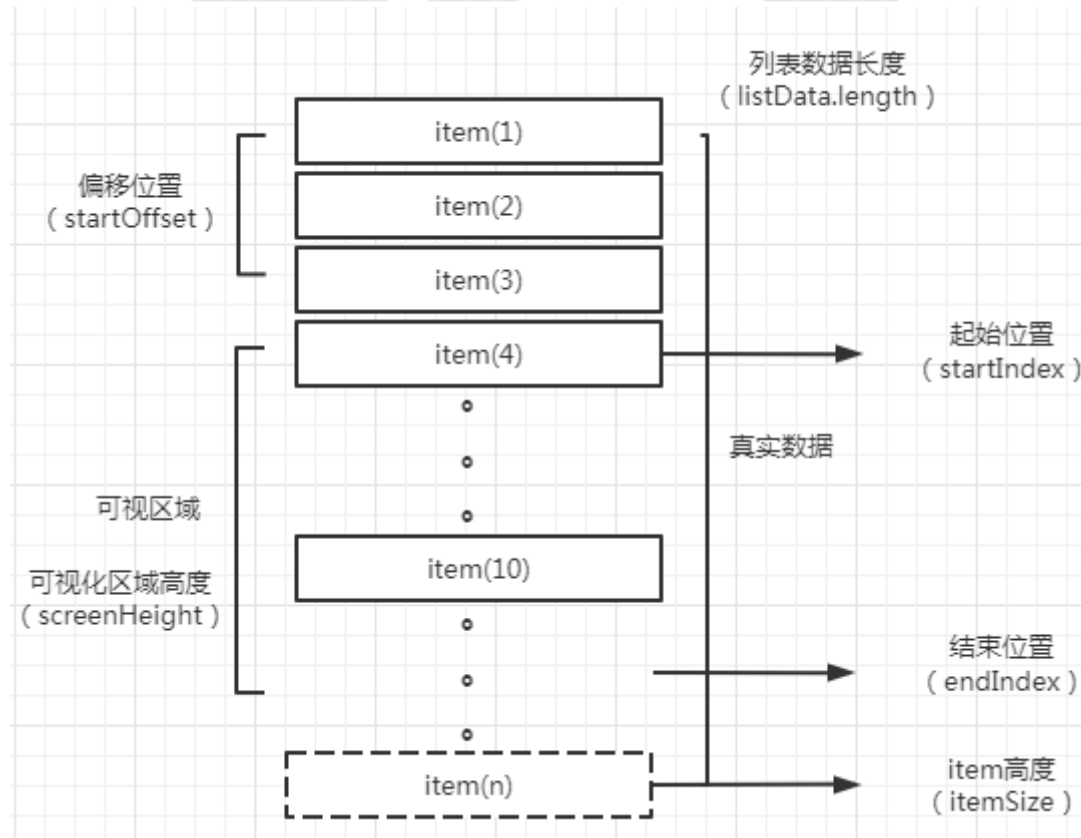
虚拟列表的实现，实际上就是在首屏加载的时候，只加载 可视区域 内需要的列表项，当滚动发生时，动态通过计算获得 可视区域 内的列表项，并将 非可视区域 内存在的列表项删除。



5、页面结构

```
//可视化区域容器
<div class="list-container">
  //真实数据容器，以便生成滚动条
  <div class="list-phantom"></div>
  //渲染区域
  <div class="list">
    <!-- item(1) -->
    <!-- item(2) -->
    <!-- ..... -->
    <!-- item(3) -->
  </div>
</div>
```

6、接着，监听 list-container 的 scroll 事件，获取滚动位置 scrollTop



推算出：

- 列表总高度 `listHeight` = `listData.length * itemSize`
- 可显示的列表项数 `visibleCount` = `Math.ceil(screenHeight / itemSize)`
- 数据的起始索引 `startIndex` = `Math.floor(scrollTop / itemSize)`
- 数据的结束索引 `endIndex` = `startIndex + visibleCount`
- 列表显示数据为 `visibleData` = `listData.slice(startIndex, endIndex)`
- 偏移量 `startOffset` = `scrollTop - (scrollTop % itemSize);`

7、完整版

```
<template>
  <div ref="list" class="list-container" @scroll="scrollEvent($event)">
    <div class="list-phantom" :style="{ height: listHeight + 'px' }"></div>
    <div class="list" :style="{ transform: getTransform }">
      <div ref="items"
        class="list-item"
        v-for="item in visibleData"
        :key="item.id"
        :style="{ height: itemSize + 'px', lineHeight: itemSize + 'px' }"
        >{{ item.value }}</div>
    </div>
  </div>
</template>
```

```
export default {
  name: 'VirtualList',
  props: {
    //所有列表数据
    listData: {
      type: Array,
      default: () => []
    }
  }
}
```

```

    },
    //每项高度
    itemSize: {
      type: Number,
      default: 200
    }
  },
  computed: {
    //列表总高度
    listHeight() {
      return this.listData.length * this.itemSize;
    },
    //可显示的列表项数
    visibleCount() {
      return Math.ceil(this.screenHeight / this.itemSize)
    },
    //偏移量对应的 style
    getTransform() {
      return `translate3d(0, ${this.startOffset}px, 0)`;
    },
    //获取真实显示列表数据
    visibleData() {
      return this.listData.slice(this.start,
        Math.min(this.end, this.listData.length));
    }
  },
  mounted() {
    this.screenHeight = this.$el.clientHeight;
    this.start = 0;
    this.end = this.start + this.visibleCount;
  },
  data() {
    return {
      //可视区域高度
      screenHeight: 0,
      //偏移量
      startOffset: 0,
      //起始索引
      start: 0,
      //结束索引
      end: null,
    };
  },
  methods: {
    scrollEvent() {
      //当前滚动位置
      let scrollTop = this.$refs.list.scrollTop;
      //此时的开始索引
      this.start = Math.floor(scrollTop / this.itemSize);
      //此时的结束索引
      this.end = this.start + this.visibleCount;
      //此时的偏移量
      this.startOffset = scrollTop - (scrollTop % this.itemSize);
    }
  }
};

```

